Journal of Network and Computer Applications 245 (2026) 104398



Contents lists available at ScienceDirect

Journal of Network and Computer Applications

journal homepage: www.elsevier.com/locate/jnca



Research paper



# A quantum-secure digital signature-based communication protocol for the Internet of Drones (IoD)

Jithu Vijay V.P. *, Shahanas I.N., Sabu M. Thampi, Aiswarya S. Nair

*Connected Systems and Intelligence Lab, School of Computer Science and Engineering (SoCSE), Digital University Kerala, Technopark Phase IV, Thiruvananthapuram, 695317, Kerala, India*

ARTICLE INFO

*Keywords:*
Drones
Unmanned Aerial Vehicles (UAVs)
Internet of Drones (IoD)
Secure communication
Post Quantum Cryptography (PQC)
Kyber
Dilithium
Physical Unclonable Functions (PUF)
Hyperledger fabric blockchain

ABSTRACT

The use of drones is rapidly increasing in areas such as surveillance, defense, and emergency services. As a result, ensuring secure communication and proper authentication has become a major concern in the Internet of Drones, where drones must share data and coordinate their actions in real time. One of the biggest challenges in drone networks is maintaining secure and reliable communication between drones. The dynamic and distributed nature of these networks increases the risk of security breaches. Existing systems mostly rely on cryptographic methods like RSA and ECC. These methods will not remain secure in the future because of advancements in quantum computing. These systems also depend on static data storage and centralized credential management, which make them vulnerable to attacks such as impersonation, replay, and man-in-the-middle. To address these issues, we propose a quantum-secure drone-to-drone authentication and secure communication protocol that utilizes Post Quantum Cryptographic (PQC) algorithms such as, Kyber for encryption and Dilithium for digital signatures. Both are lattice-based lightweight cryptographic algorithms that offer strong resistance against quantum attacks. Instead of storing secret data on drones and to prevent cloning, we use Physical Unclonable Functions (PUF) to generate device specific seeds for authentication and key generation during each session. A Hyperledger Fabric Blockchain is used at the Ground Control Station (GCS) to store drone credentials securely and avoid single point failure. We conducted the formal security analysis using the Burrows–Abadi–Needham (BAN) logic for trust validation and the Scyther tool to formally analyze and verify resistance against classical and quantum-era attacks. In addition to formal proofs, informal analysis confirms that the protocol maintains data integrity and authentication even under active network threats. We implemented the protocol using Raspberry Pi drones and a Linux-based GCS. Performance results show a low computation time of 0.08 s for authentication and 0.12 s for secure communication on Raspberry Pi 5, with minimal memory usage and acceptable communication cost suitable for implementation on resource-constrained drones.

## 1. Introduction

The Internet of Drones (IoD) is revolutionizing modern applications by enabling interconnected operations among Unmanned Aerial Vehicles (UAVs), commonly known as drones. As these devices shift from standalone systems to dynamic, collaborative networks, IoD plays a major role in improving communication, data sharing, and coordination (Boccadoro et al., 2021; Alzahrani et al., 2020). Whether in disaster response, military surveillance, or logistics, drones rely heavily on seamless communication with the Ground Control Station (GCS) and other drones to achieve operational success. However, these communication systems are increasingly becoming prime targets for cyberattacks. This is mainly because of their wireless nature and the sensitive information they transmit (Derhab et al., 2023; Hadi et al., 2023; Nair et al., 2025; Jithu Vijay and Thampi, 2025).

The security of IoD communication protocols is crucial due to the highly sensitive nature of transmitted information, including live surveillance data, operational commands, and mission-critical details. Traditional security mechanisms are proving inadequate against emerging threats. A recently identified vulnerability in DJI UAVs allowed attackers to intercept and manipulate control commands, providing remote drone hijacking through weaknesses in the Enhanced Wi-Fi protocol (Pratama et al., 2024). UAV administrators typically handle sensitive data such as flight history, photos captured during operations, real-time UAV camera access rights, and location information. Consequently, security breaches targeting these devices lead to the

* Corresponding author.
*E-mail address:* jithu.csres22@duk.ac.in (Jithu Vijay V.P.).

https://doi.org/10.1016/j.jnca.2025.104398
Received 1 July 2025; Received in revised form 27 October 2025; Accepted 22 November 2025
Available online 26 November 2025
1084-8045/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

exposure of sensitive user data, thereby compromising both security and privacy.

Apart from such real incidents, other types of cyber attacks have also been seen in IoD communication systems (Yaacoub et al., 2020). Attackers have taken advantage of weak encryption and authentication to perform Man-in-the-Middle (MitM) attacks, alter data, and inject unauthorized commands. In military applications, such breaches can expose sensitive locations and compromise intelligence operations, while in disaster response, they could disrupt rescue efforts.

A major challenge in IoD communication is ensuring the confidentiality, integrity, and availability of data. Although traditional cryptographic techniques such as encryption and digital signatures have secured communication, the rise of quantum computing poses a new threat. Quantum computers have the capability to break widely used cryptographic methods such as RSA (Rivest Shamir Adleman) and ECC (Elliptic Curve Cryptography). This makes the current security protocols inadequate (Subramani et al., 2025; Tambe-Jagtap, 2023; Burhanuddin, 2023). To secure IoD systems in the future, the adoption of quantum-resistant cryptographic solutions is essential for long-term protection (Singamaneni and Muhammad, 2024).

Recent studies have explored using Post Quantum Cryptography (PQC) to improve the security of drones and IoT systems. Algorithms such as Kyber, Dilithium, ROLLO, and BIKE have been applied in lightweight environments to protect against future quantum threats (Abbood et al., 2025; Khan et al., 2024; Sharma and Rani, 2025; Mansoor et al., 2024a; Duraibi and Alashjaee, 2025; Chen et al., 2024). These approaches help secure key exchange, authentication, and end-to-end communication even on devices with limited resources. However, many of these solutions still store keys in device memory, which makes them vulnerable to physical attacks such as cloning and side-channel attacks. Also, most existing works do not focus on managing credentials securely in changing drone networks.

Some recent works have focused on applying PQC in other smart environments such as transportation systems, smart grids, and drone-based services. They mostly use lattice-based and code-based encryption to provide better privacy and authentication (Mansoor et al., 2024b; Parameswarath et al., 2025; Ojetunde et al., 2025; Prajapat et al., 2024; Tawfeeq et al., 2025). These works show progress in reducing computation load and applying PQC in network setups. But they are mostly done in simulations and virtual setups, and not tested much on actual hardware such as Raspberry Pi and drones. Also, features such as dynamic key generation, secure credential storage, and scaling up to larger IoD swarms are often missing.

Drones, with their limited computational resources, memory, and battery life, require lightweight security solutions. Unlike other devices, drones operate in environments where heavy cryptographic tasks can quickly drain power and disrupt real-time communication. Therefore, it is crucial to design IoD communication protocols that maintain security without compromising resource efficiency.

### 1.1. Motivation

As IoD networks become more widespread, maintaining secure communication between drones has become increasingly critical. Communication is not limited to basic information exchange; it is crucial for mission-critical operations such as surveillance, disaster response, military missions, and logistics. Ensuring secure communication involves addressing key security principles: confidentiality, integrity, authentication, and non-repudiation. Consider a military sector where IoD systems are deployed for battlefield operations. In such scenarios, ensuring the confidentiality and integrity of communication is critical, as real-time commands and surveillance data are continuously exchanged.

Despite utilizing cryptographic solutions to counter these vulnerabilities, UAV systems remain exposed to various security threats, including eavesdropping, network jamming, weak authentication protocols, and challenges in mobility management (Wang et al., 2024; Wei et al., 2024). These threats increase the risks in UAV networks with the growing reliance on wireless communication. Also, due to their limited battery life and computational capacity, drones cannot afford the overhead of resource-intensive cryptographic protocols (Zhao et al., 2025). To keep communication secure, drones need a way to authenticate each other without relying on permanently stored keys, which can be physically tampered with. In addition, in widely distributed drone networks, relying on a central authority can lead to system failure.

The rapid advancements in quantum computing further increase security concerns for IoD systems. The majority of existing cryptographic techniques are based on two popular algorithms, such as RSA and ECC. Unlike classical computers, which require infeasible amounts of time to break these encryption methods through brute force, quantum computers employing Shor's Algorithm can perform the same tasks exponentially faster, making conventional cryptographic techniques ineffective (Kumar and Mondal, 2024). RSA relies on the difficulty of factoring large composite numbers, while ECC depends on the hardness of solving the discrete logarithm problem over elliptic curves. Shor's Algorithm can solve both problems in polynomial time, which classical algorithms are not able to do efficiently.

As quantum computers continue to evolve, their increasing computational power poses a threat to IoD systems, which often handle highly sensitive data and require strong communication security. In high-risk environments, such as a drone that relays control information to a defense operation center, even a short interception or unauthorized command injection could take control of the drones. Implementing quantum-secure communication protocols in military IoD systems ensures that even if an adversary possesses a quantum computer, they cannot decode messages or impersonate trusted drones. This is crucial in operations where drones are deployed across various domains and controlled via satellite links in contested airspace. Therefore, developing a decentralized quantum-secure communication protocol is important not just for the future. It also helps keep military operations secret and prevents attackers from altering drone commands in dangerous situations.

### 1.2. Contributions

In this paper, we present a quantum-secure drone-to-drone authentication and communication protocol for the IoD, which combines the Kyber encryption scheme, Dilithium-based digital signatures, Physical Unclonable Functions (PUF), and a permissioned blockchain (Hyperledger Fabric). This comprehensive framework ensures secure, lightweight, and efficient communication in IoD systems, addressing critical challenges posed by existing security mechanisms.

Kyber (officially CRYSTALS-Kyber), a lattice-based cryptographic algorithm, provides quantum-resistant encryption, protecting sensitive data from interception and tampering (Nguyen et al., 2025). It is based on the Module-Learning With Errors (M-LWE) problem, which ensures computational infeasibility even for quantum computers. Recognized as one of the winning candidates in the NIST PQC Standardization Process, Kyber provides strong security and performance. Its lightweight nature allows encryption to operate efficiently without compromising real-time communication.

Similarly, Dilithium (officially CRYSTALS-Dilithium), another lattice-based cryptographic algorithm, is employed for message authentication and integrity verification (Shen et al., 2024). Dilithium is based on the Learning With Errors (LWE) and Short Integer Solution (SIS) problems and provides robust digital signature capabilities. It ensures that transmitted messages are tamper-proof and originate from authenticated sources. It is also selected as one of the winning candidates in the NIST PQC Standardization Process. Dilithium creates small-sized digital signatures and allows fast verification, which makes it a good choice for large IoD systems where many drones need to communicate securely at the same time.

PUF are utilized in the framework to generate unique response for a given challenge during the authentication and Kyber–Dilithium key generation process. Unlike traditional methods that rely on stored cryptographic keys, PUF exploit the inherent physical variations in drone hardware to produce device-specific and unpredictable responses to the given challenge (Tanveer et al., 2024). This approach ensures robust identity verification and unique key generation. Each drone's response is intrinsically tied to its physical hardware, making it resistant to cloning and tampering. The use of PUF minimizes vulnerabilities such as cloning or spoofing attacks.

To manage drone credentials securely, the framework utilizes a permissioned blockchain implemented with Hyperledger Fabric. Blockchain technology ensures decentralized, tamper-proof, and immutable storage of credentials during the drone registration process at the GCS. Hyperledger Fabric's channel-based privacy and pluggable architecture useful for managing and verifying credentials efficiently (Vinoth et al., 2024). In the proposed framework, Kyber encryption is used to secure credential data stored on the blockchain, ensuring confidentiality even in the face of quantum computing threats.

Finally, the proposed framework undergoes a comprehensive security and performance analysis to validate its strength against a broad spectrum of attacks, including replay attacks, tampering, MitM attacks, and impersonation attempts. The performance evaluation demonstrates that the protocol is both efficient and scalable, ensuring minimal computational overhead and energy consumption. By addressing these challenges, the proposed framework delivers a secure and effective solution for protecting communication within the IoD. The main contributions are listed below:

1. We have proposed the first quantum-secure drone-to-drone authentication and communication protocol utilizing PQC techniques such as Kyber and Dilithium, PUF, and Hyperledger Fabric.
2. Designed a permissioned blockchain system using Hyperledger Fabric to securely store and manage drone credentials during the registration process at the GCS, providing decentralized and immutable data management.
3. The communication protocol is also implemented on a real UAV (powered by Raspberry Pi) and a Linux-based GCS using a WiFi Network.
4. Conducted comprehensive security and performance analysis, showing the framework's resistance to replay attacks, impersonation, tampering, and MitM attacks while maintaining lightweight and scalable performance suitable for resource-constrained drones.

The paper is structured in nine sections. Section 2 deal with related works. The network model and threat model are presented in the subsequent section, followed by the proposed methodology, algorithmic details of the proposed system, security analysis, implementation details, and performance analysis and results. Finally, the paper is concluded in Section 9.

## 2. Related works

This section reviews state-of-the-art communication and authentication schemes within IoD, focusing on their limitations in quantum resistance, scalability, and physical security. While significant progress has been made in addressing some of these challenges, this review identifies main research gaps that underline the need for robust solutions.

Zhang et al. (2020) proposed a lightweight authentication and key agreement (AKA) scheme using one-way hash functions and XOR operations. However, it fails to ensure anonymity protections and is prone to physical attacks, which makes it unsuitable for secure IoD applications. Tian et al. (2019) introduced an edge-assisted privacy-preserving authentication framework. While pseudonym updates improve privacy, the scheme incurs high communication costs and lacks quantum-resistant cryptography. Similarly, Srinivas et al. (2019) developed TCALAS, a temporal credential-based lightweight authentication protocol. Despite resisting conventional attacks, it is vulnerable to device compromise and lacks quantum resilience.

Wazid et al. (2018) presented a taxonomy and analysis of IoD authentication protocols. Their ECC-based lightweight protocol is computationally efficient, but does not address quantum security, limiting its applicability in future IoD networks. Hussain et al. (2021) proposed an ECC-based authentication scheme focused on energy efficiency. However, reliance on ECC makes it vulnerable to quantum computing threats, and it does not address drone capture attacks. Yahuza et al. (2021) introduced the SLPAKA certificateless lightweight authentication protocol for scalable IoD deployments. Although addressing scalability, the scheme is computationally intensive for resource-constrained drones and lacks quantum resistance.

Nikooghadam et al. (2021) designed an ECC-based protocol for secure smart city surveillance applications in IoD. While it ensures low computational costs, it lacks scalability and robustness against insider attacks. Zhang et al. (2021) proposed RUAM-IoD, an elliptic curve-based authentication scheme that ensures secure key exchanges. Despite being robust against cyberattacks, it struggles with quantum resilience and scalability. Wazid et al. (2019) introduced a multifactor authentication scheme utilizing dynamic pseudonyms. However, it demonstrates limited scalability and does not account for evolving quantum threats.

Tanveer et al. (2022) presented an efficient identity management scheme for IoD. Although it supports privacy and security, it remains vulnerable to physical attacks due to static key storage. El-Zawawy et al. (2022) developed SETCAP, a service-based temporal credential authentication protocol. While reducing communication costs, it relies on classical cryptography, leaving it vulnerable to quantum attacks. Pu et al. (2022) proposed PMAP, a lightweight and privacy-preserving authentication protocol. Despite strong physical security, it is computationally intensive for large-scale IoD deployments.

Yu et al. (2022) introduced SLAP-IoD, a PUF-based lightweight authentication protocol designed for smart city environments. While secure, it struggles with dynamic IoD network integration. Alladi et al. (2020) developed SecAuthUAV, a PUF-based protocol addressing UAV-to-UAV and UAV-to-GS communication. However, its reliance on static challenge–response pairs limits adaptability in dynamic scenarios. Seo et al. (2016) proposed a white-box cryptography-based framework for secure delivery drones. While effective in resource-limited environments, scalability remains unaddressed.

Kriz and Gabrlik (2015) introduced UranusLink, a communication protocol enabling low-latency, secure UAV-to-base station communication. However, it lacks quantum resistance and physical attack countermeasures. Won et al. (2015) presented eCLSC-TKEM, a certificateless signcryption protocol that supports authenticated key exchange. Although efficient, its reliance on classical cryptographic primitives leaves it vulnerable to quantum attacks.

Islam et al. (2016) proposed a group key establishment protocol for FANETs, enabling fast and secure node communication. However, the protocol struggles with scalability in high-mobility scenarios. Maxa et al. (2016) verified UAANET routing protocols, highlighting secure communication features, but failing to address vulnerabilities to quantum cryptographic threats. Blazy et al. (2017) proposed a protocol ensuring UAV communication confidentiality using time-evolving keys. Although robust, its reliance on symmetric keys makes it susceptible to quantum decryption techniques.

Wang et al. (2017) developed a handover key management scheme for LTE-based UAV control networks, addressing frequent key updates. However, it lacks quantum-resistant mechanisms that pose risks in the future. Semal et al. (2018) introduced a certificate-less group-authenticated key agreement protocol for UAV networks. While it ensures secure communication in untrusted environments, scalability

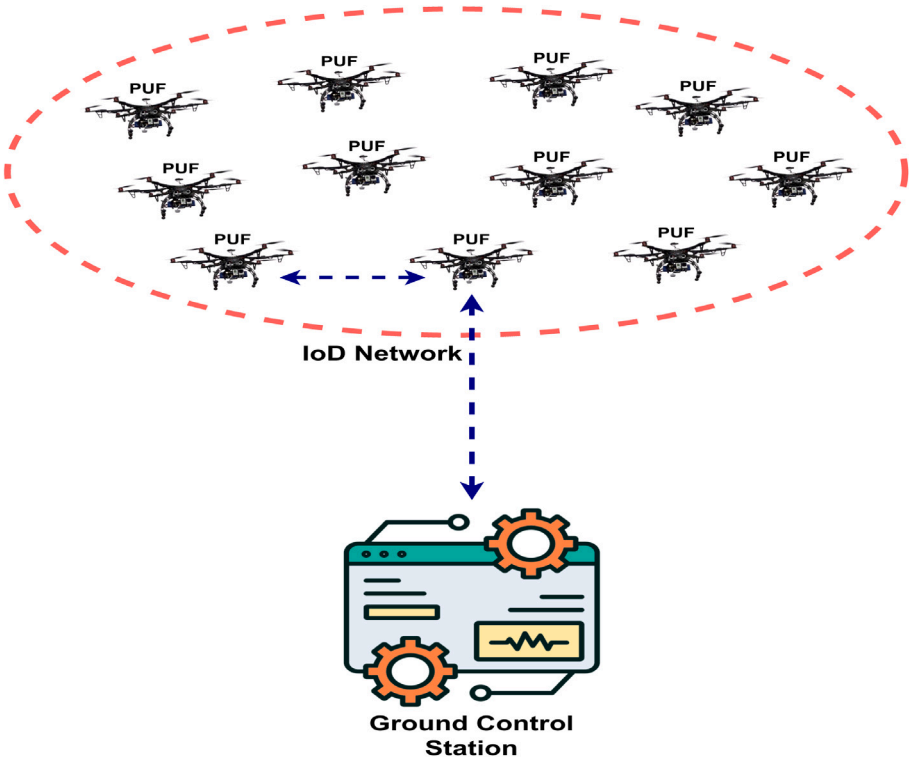



**Fig. 1.** Network model of the proposed communication protocol in IoD.

and computational overhead remain concerns. Ding et al. (2017) presented a post-quantum PAKE protocol using Ring-Learning With Errors (R-LWE). Although quantum resistant, its computational demands are impractical for resource-limited UAVs.

Mishra et al. (2023) proposed a quantum-safe communication scheme for IoD based on lattice cryptography. The scheme offers robust security against quantum threats while maintaining efficiency. However, reliance on specific hardware environments restricts its widespread applicability. Ko et al. (2021) proposed a secure communication protocol for military UAVs. The protocol includes two sub-protocols: SP-D2GCS, designed to secure communication between drones and GCS, and SP-D2MD, which ensures secure interactions between drones and monitoring drones. Both protocols rely on mutual authentication and hierarchical communication, but impose high computational overhead during initial key exchanges. Also, time synchronization requirements limit scalability in highly dynamic IoD environments. Anon (2025) present a systematic review of integrating Quantum Key Distribution (QKD) into 6G infrastructures and demonstrate its ability to provide security against quantum attacks. They discuss hybrid models combining QKD and PQC, as well as the role of AI in improving key distribution and error correction. These developments show how future networks are actively exploring quantum-safe mechanisms to make communication more secure at the infrastructure level. The summary of major works in communication protocols is shown in Table 1. However, none of these works in literature offers a comprehensive solution that ensures quantum resistance, scalability, low computational overhead, physical security against drone capture, and adaptability to the dynamic and resource-constrained nature of the IoD.

The analysis of existing communication and authentication protocols for IoD reveals several critical gaps that remain unaddressed, especially in the context of quantum resistance, scalability, and physical security. While advancements such as blockchain-based frameworks and PUF-based authentication schemes have significantly improved security and privacy, many solutions, including those by Yahuza et al. (2021) and Pu et al. (2022), struggle with the computational demands of large-scale IoD deployments. Furthermore, reliance on ECC and RSA in several approaches, as mentioned in the works of Wazid et al. (2019) and Tanveer et al. (2022), exposes them to quantum attacks. Although blockchain systems such as those proposed by Zhang et al. (2021) and Wang et al. (2017) offer decentralized credential management, they face challenges such as, high communication overhead and limited scalability. To overcome these limitations, our proposed framework integrates PQC algorithms, lightweight PUF-based authentication and key generation, and a permissioned blockchain to deliver a quantum-secure, scalable, and resource-efficient solution for IoD communication and authentication. This approach effectively addresses the gaps identified in existing literature and ensures robustness against evolving security threats.

## 3. Network and threat model

The network model comprises a GCS, which integrates a private blockchain network to store drone credentials securely and in a tamper-proof manner. The GCS oversees a fleet of drones, as illustrated in Fig. 1. Upon successful registration, the GCS maintains the credentials of all drones (including each drone's ID, PUF challenge–response pair, Kyber public key, and Dilithium public key) in the network. Each drone is equipped with a PUF module. For each session, both Dilithium (for digital signatures) and Kyber (for encryption) key pairs are derived from a seed value generated via the PUF. As a result, secret keys do not need to be stored on the drone, reducing the risk of key compromise. When a drone initiates communication with another drone, the receiving drone first contacts the GCS to retrieve the credentials of the requesting drone. Before releasing these credentials, the GCS verifies the identity of the requesting drone. Only upon successful verification does the GCS transmit the necessary credentials. Following this, the receiving drone authenticates the requester and proceeds to establish a secure communication session.

To formally assess the security of the proposed architecture, we adopt the Dolev–Yao threat model, which represents a strong adversarial assumption widely used in protocol analysis. Under this model, the adversary is assumed to have complete control over the communication network. This includes the ability to intercept, eavesdrop,

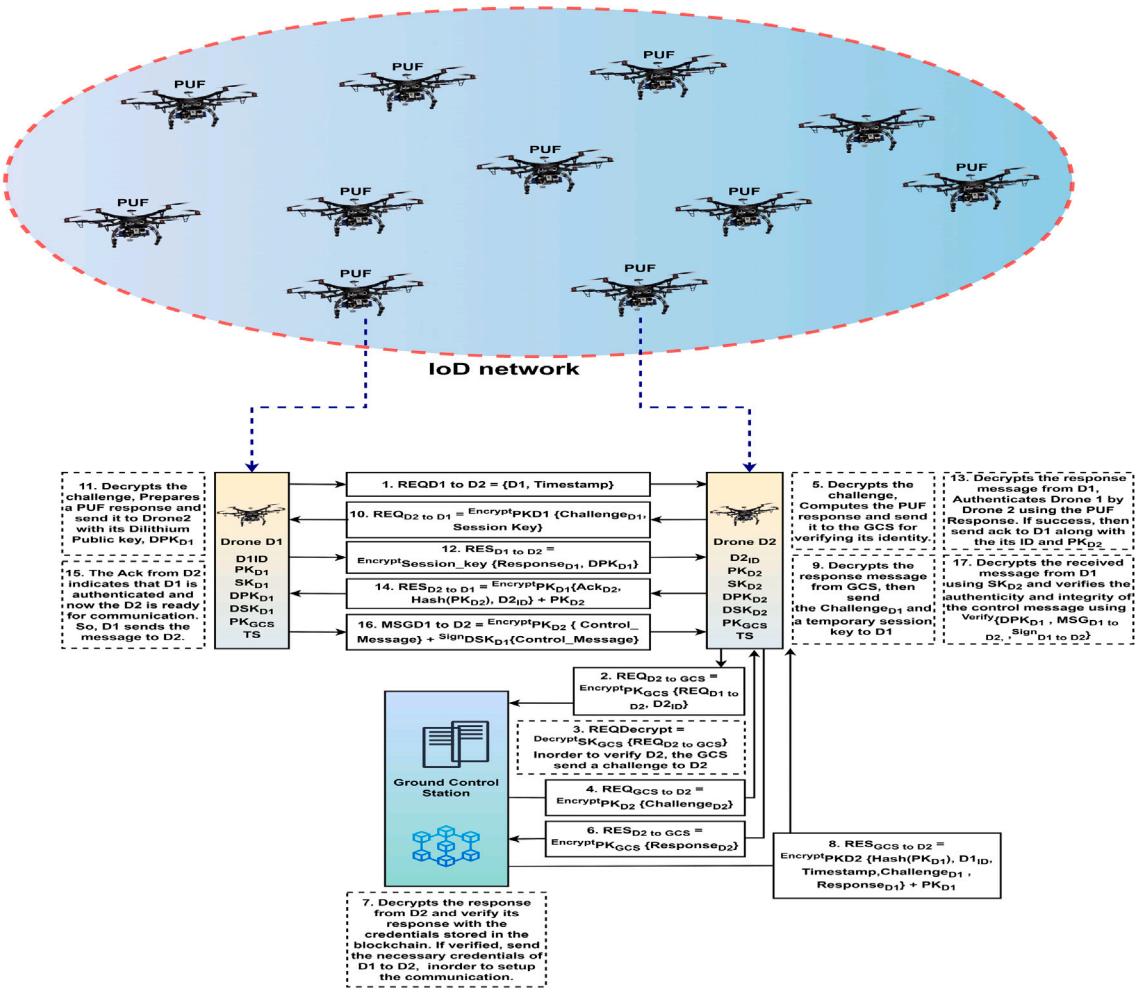



**Fig. 2.** Workflow of proposed drone to drone authentication and secure communication protocol for the IoD.

**Table 1**
Summary of important communication protocols for IoD.

| Author | Network | Technique | Issues |
|---|---|---|---|
| Zhang et al. (2020) | IoD | Lightweight AKA with one-way hash and XOR | Lacks anonymity, vulnerable to physical attacks, not quantum-resistant |
| Tian et al. (2019) | IoD | Edge-assisted privacy-preserving framework | High communication costs, not quantum-resistant |
| Yahuza et al. (2021) | IoD | SLPAKA (Certificateless Lightweight Protocol) | Computationally intensive for resource-constrained drones, lacks quantum resistance |
| Pu et al. (2022) | IoD | PMAP (Lightweight Privacy-Preserving Protocol) | Strong physical security but computationally intensive, not quantum-resistant |
| Yu et al. (2022) | IoD | SLAP-IoD (PUF-Based Lightweight Protocol) | Struggles with dynamic network integration, not quantum-resistant |
| Blazy et al. (2017) | IoD | Protocol with time-evolving keys | Strong but vulnerable to quantum decryption |
| Wang et al. (2017) | IoD | Handover key management scheme | Lacks quantum-resistant mechanisms, scalability issues |
| Ding et al. (2017) | IoD | Post-Quantum PAKE (Ring-Learning-With-Errors) | Quantum-resistant but inefficient for resource-constrained drones |
| Ko et al. (2021) | IoD | SP-D2GCS and SP-D2MD | High computational overhead during key exchanges, not quantum-resistant |

replay, modify, block, and fabricate messages arbitrarily. However, it is also assumed that cryptographic primitives, such as encryption and digital signatures, are ideal and cannot be broken without the corresponding secret keys. This abstraction allows protocol-level vulnerabilities to be isolated and analyzed rigorously without conflating them with weaknesses in cryptographic implementation. In the context of our system, this model captures various real-world attack vectors such as impersonation, replay attacks, MitM intrusions, and key compromise attempts. By employing PUF-generated session-specific key pairs and post-quantum algorithms (Dilithium and Kyber), our protocol ensures that even a Dolev–Yao adversary cannot subvert authentication or decrypt messages without access to the ephemeral keys, which are never stored and are regenerated afresh for each session. This makes the architecture highly resilient to both conventional and quantum-capable attackers operating under a Dolev–Yao threat environment.

## 4. Proposed methodology

We are proposing a quantum-secure digital-signature based drone-to-drone authentication and communication protocol for IoD to address the research gaps in the existing communication systems, using lightweight PQC, PUF, and Blockchain. The workflow of the proposed system is shown in Fig. 2. We have an IoD network, connected with the GCS. The GCS utilizes Hyperledger Fabric blockchain to store the drone credentials, and each drone in the IoD network is PUF-enabled. To avoid static key storage in drones, Kyber and Dilithium keys are generated for each session using a device-specific seed derived from the PUF, as illustrated in algorithms 1 and 2. Initially, all drones (Drone 1, Drone 2, Drone 3, ….., Drone n) are registered with the GCS, after successful registration, each drone will get the GCS Kyber public key, and it will be added to the IoD network. The blockchain in the GCS store each drone's ID, PUF challenge–response pair, Kyber public key, and Dilithium public key.

Here the Drone 1 wants to communicate with Drone 2 and the process starts with Drone 1 establishing a connection by transmitting a request to Drone 2, and then Drone 2 transmits it to the GCS using Kyber encryption with the GCS's public key. The GCS verifies the authenticity of Drone 2 and shares the necessary credentials of Drone 1 to further authenticate by Drone 2. After authentication between Drone 1 and Drone 2 using Kyber encryption and Dilithium digital signatures, subsequent communications between the drones are encrypted and signed, so that authenticity and integrity are maintained in the drone operations. Detailed description of the workflow of our proposed system is explained as follows:

- **Drone 1 initiates connection:** Drone 1 begins the communication process by sending a request containing its ID and timestamp to Drone 2. This initial request serves as a handshake, indicating Drone 1's intent to establish communication with Drone 2.
- **Forwarding to GCS:** Upon receiving the request, Drone 2 forwards it to the GCS, along with its own ID and timestamp. The information is encrypted using the GCS's public key through Kyber encryption method, as shown in algorithm 3. By encrypting the message with the GCS's public key, Drone 2 ensures that only the GCS, possessing the corresponding private key, can decrypt and access the contents of the request.
- **Verification by GCS:** The GCS decrypts the received request using its private key through Kyber decryption method, as shown in algorithm 4. Upon decryption, the GCS verifies the authenticity of Drone 2 by sending the challenge encrypted using the Kyber public key of Drone 2 and Drone 2 sends the corresponding PUF response encrypted using the public key of GCS, as shown in algorithm 3). Then, the GCS decrypts the response from Drone 2 using Algorithm 4 and verifies it against the PUF response stored in the blockchain (during registration). Successful verification allows the process to proceed to the next stage.
- **challenge–response Exchange:** Once Drone 2's authenticity is confirmed, the GCS responds by sending an encrypted message (using algorithm 3 concatenated with Drone 1's Kyber public key to Drone 2. This encrypted message contains the hash of Drone 1's Kyber public key and a challenge–response pair. The PUF based challenge–response mechanism ensures verification of Drone 1's identity and helps prevent unauthorized access.
- **Challenge Propagation:** Drone 2 decrypts the message (using algorithm 4 from the GCS through its private key and extracts Drone 1's challenge. Drone 2 then computes the hash of the received Kyber pubic key of Drone 1 and compares it with the hash decrypted from the received encrypted message. If successful, the challenge is forwarded to Drone 1 along with a new session key (derived from the PUF using a random challenge), both encrypted using Drone 1's public key, as described in Algorithm 3.
- **Response and Authentication:** Drone 1 receives the challenge from Drone 2, decrypts through its private key (using algorithm 4) and computes the PUF response. This response, along with the Drone 1's Dilithium public key encrypted through the session key obtained from the previous message is sent back to Drone 2. Drone 2 decrypts this message using the session key, and authenticates Drone 1 by comparing the received PUF response with the one forwarded by the GCS. If it is successful, then Drone 2 sends an encrypted message (using algorithm 3) concatenated with Drone 2's Kyber public key to Drone 1. This encrypted message contains the acknowledgment, hash of the Drone 2's Kyber public key, and ID of Drone 2. The acknowledgment from Drone 2 indicates that Drone 1 is authenticated and now it is ready for communication.
- **Subsequent Communication:** For subsequent communications, Drone 1 signs control messages using the Dilithium digital signature scheme, as shown in algorithm 5 and encrypts the original control message through Drone 2's Kyber public key (using algorithm 3). Upon receiving these encrypted messages, Drone 2 decrypts the control message through its private key (using algorithm 4) and verifies the signature using Drone 1's Dilithium public key, as shown in algorithm 6. Successful verification confirms the authenticity and integrity of the received message, allowing for secure and trusted communication between the drones.

## 5. Algorithmic details of the proposed system

The algorithm 1 describes the PUF-based kyber key generation at drone. Upon receiving a challenge, the drone's Static Random Access Memory (SRAM) PUF generates a 32-bit response, denoted as $d$. Since this output is insufficient in size and entropy for cryptographic use on its own, it is expanded using the SHAKE-128 hash function. To ensure separation between cryptographic domains, two distinct inputs are passed through SHAKE-128: $d \parallel 0x00$ and $d \parallel 0x01$, producing two independent 32-byte values. The first output, $\rho$, serves as the seed for generating the public matrix $\hat{A}$, while the second, $\sigma$, acts as the seed for a pseudorandom function (PRF) used in noise sampling. A nonce counter $N$ is initialized and incremented to ensure unique PRF input for each coefficient. Using the PRF seeded with $\sigma$, the secret vector $s$ and error vector $e$ are sampled through a Centered Binomial Distribution $\text{CBD}_\eta$. These vectors are then transformed into the Number Theoretic Transform (NTT) domain for efficient polynomial arithmetic. The public component is computed as $\hat{t} = \hat{A} \odot \hat{s} + \hat{e}$, where $\odot$ denotes matrix–vector multiplication in the NTT domain. Finally, the public key $p_k$ is formed by encoding $\hat{t}$ together with the seed $\rho$, and the secret key $s_k$ is derived by encoding the vector $s$.

**Algorithm 1** PUF-based Kyber Key Generation at Drone

**Input:** Challenge **Output:** $s_k, p_k$

$d \leftarrow \text{SRAMPUF}(\text{Challenge})$
$\rho \leftarrow \text{SHAKE-128}(d \| 0x00)[:32]$
$\sigma \leftarrow \text{SHAKE-128}(d \| 0x01)[:32]$
$N \leftarrow 0$
$\hat{A} \leftarrow \text{GenMatrix}(\rho)$
**for** $i = 0$ to $k-1$ **do**
  $s[i] \leftarrow \text{CBD}_\eta(\text{PRF}(\sigma, N))$
  $N \leftarrow N + 1$
**end for**
**for** $i = 0$ to $k-1$ **do**
  $e[i] \leftarrow \text{CBD}_\eta(\text{PRF}(\sigma, N))$
  $N \leftarrow N + 1$
**end for**
$\hat{s} \leftarrow \text{NTT}(s)$
$\hat{e} \leftarrow \text{NTT}(e)$
$\hat{t} \leftarrow \hat{A} \odot \hat{s} + \hat{e}$
$p_k \leftarrow \text{Encode}(\hat{t} \| \rho)$
$s_k \leftarrow \text{Encode}(s)$

**Table 2**
BAN logic symbols and interpretations.

| Symbol | Meaning |
|---|---|
| $R \mid\equiv K$ | Principal $R$ believes that the message or statement $K$ is valid. |
| $R \triangleleft K$ | $R$ has received and can interpret the message $K$. |
| $R \mid\sim K$ | $R$ previously generated and sent message $K$. |
| $R \Rightarrow K$ | $R$ has authority over the correctness or origin of $K$. |
| $\#(K)$ | $K$ is fresh—generated recently and not a replayed message. |
| $\langle K \rangle_M$ | Message $K$ is bundled with message $M$. |
| $\{K\}_M$ | $K$ is encrypted using the key $M$. |
| $R \overset{m}{\leftrightarrow} M$ | $R$ and $M$ share a symmetric key $m$. |
| $R \overset{m}{\rightleftharpoons} M$ | $R$ and $M$ know a secret $m$, unknown to others. |
| $\overset{m}{\mapsto} M$ | $m$ is the public key of principal $M$. |

The algorithm 2 describes the PUF-based Dilithium key generation at drone. Upon receiving a challenge, the SRAM PUF generates a 32-bit response denoted as $d$, which serves as the foundational entropy source. Due to its limited bit length, the response is expanded using SHAKE-128 hash function. To ensure separation of cryptographic roles, domain-separated inputs to SHAKE-128 generate three independent 32-byte values: $\rho$, $\rho'$, and $K$. These values are used respectively for public matrix generation, secret vector sampling, and signature hash personalization. Using the seed $\rho$, the public matrix $\hat{A}$ is generated deterministically. The secret vectors $s_1$ and $s_2$ are sampled from a centered binomial distribution using $\rho'$ and distinct nonces to avoid overlap. The intermediate polynomial $t$ is computed as $\hat{A} \cdot s_1 + s_2$, representing the commitment to the secret. This polynomial is decomposed into $t_1$ and $t_0$, allowing for efficient encoding and compression. The Dilithium public key, denoted as dpk, includes the compressed public component $t_1$ and matrix seed $\rho$. The Dilithium secret key, dsk, encapsulates the full secret state: $s_1$, $s_2$, the low bits $t_0$, the matrix seed $\rho$, and the personalization value $K$.

**Algorithm 2** PUF-based Dilithium key Generation at Drone

**Input:** Challenge **Output:** $dpk, dsk$

$d \leftarrow$ SRAMPUF(Challenge)
$\rho \leftarrow$ SHAKE-128($d \| 0x00$)[ : 32]
$\rho' \leftarrow$ SHAKE-128($d \| 0x01$)[ : 32]
$K \leftarrow$ SHAKE-128($d \| 0x02$)[ : 32]
$\hat{A} \leftarrow$ GenMatrix($\rho$)
$s_1 \leftarrow$ SampleSmall($\rho', 0$)
$s_2 \leftarrow$ SampleSmall($\rho', 1$)
$t \leftarrow \hat{A} \cdot s_1 + s_2$
$t_1, t_0 \leftarrow$ Decompose($t$)
$dpk \leftarrow$ Encode($t_1 \| \rho$)
$dsk \leftarrow$ Encode($s_1, s_2, t_0, \rho, K$)

The algorithm 3 describes Kyber encryption process, it begins by parsing the public key $p_k$ to retrieve the public matrix seed $\rho$ and the vector $\hat{t}$, which represents the compressed form of $A \cdot s + e$ from the key generation phase. Using $\rho$, the sender reconstructs the transpose of the public matrix $\hat{A}^T$, as this is required for the matrix–vector product during encryption. To protect the message and ensure semantic security, the sender generates a fresh source of randomness $r$, typically via a cryptographic random number generator or hash function. This seed $r$ is then used to generate three independent vectors through a PRF followed by a CBD: a secret ephemeral vector $r_s$, an error vector $r_e$, and an additional error term $r'_e$ used to mask the encoded message. Next, the ephemeral secret vector $r_s$ is transformed into the NTT domain, which enables efficient polynomial multiplications. The first component of the ciphertext, $\hat{u}$, is computed by multiplying $\hat{A}^T$ with $\hat{r}_s$ and adding $r_e$ in the NTT domain. The second component, $v$, is constructed by performing a pointwise multiplication of the public key vector $\hat{t}$ with $\hat{r}_s$, applying the inverse NTT, and then adding the message (which is first encoded as a polynomial) along with the noise term $r'_e$. This ensures that the message is concealed both structurally and statistically. Finally, both components $\hat{u}$ and $v$ are encoded and concatenated to form the ciphertext $ct$, which is transmitted to the recipient.

**Algorithm 3** Kyber Encryption

**Input:** Message $m$, Public Key $p_k$ **Output:** Ciphertext $ct$

Parse $p_k$ into $(\hat{t}, \rho)$
$r \leftarrow$ SHAKE-128(`random_input`)
$N \leftarrow 0$
$\hat{A}^T \leftarrow \text{GenMatrix}^T(\rho)$
**for** $i = 0$ to $k - 1$ **do**
    $r_s[i] \leftarrow \text{CBD}_\eta(\text{PRF}(r, N))$
    $N \leftarrow N + 1$
**end for**
**for** $i = 0$ to $k - 1$ **do**
    $r_e[i] \leftarrow \text{CBD}_\eta(\text{PRF}(r, N))$
    $N \leftarrow N + 1$
**end for**
$r'_e \leftarrow \text{CBD}_\eta(\text{PRF}(r, N))$
$\hat{r}_s \leftarrow \text{NTT}(r_s)$
$\hat{u} \leftarrow \hat{A}^T \odot \hat{r}_s + \text{NTT}(r_e)$
$v \leftarrow \text{InverseNTT}(\hat{t} \odot \hat{r}_s) + r'_e + \text{Encode}(m)$
$ct \leftarrow \text{Encode}(\hat{u}, v)$

The algorithm 4 describes the Kyber decryption. Upon receiving a ciphertext $ct$, the algorithm begins by parsing it into its two components: $\hat{u}$, a vector of polynomials in the NTT domain, and $v$, a masked message polynomial. These components were generated during the encryption process using the sender's ephemeral secret and noise. The secret key $s_k$, which was generated during the key generation phase, is decoded to obtain the secret vector $s$. This vector represents the private polynomial used to compute the original public key component during encryption. To perform the decryption, the algorithm first transforms the secret vector $s$ into the NTT domain, resulting in $\hat{s}$. This is necessary because $\hat{u}$, received as part of the ciphertext, is also in the NTT domain, and polynomial multiplication in this domain is significantly more efficient. The decryption core involves computing the polynomial $v - \text{NTT}^{-1}(\hat{u} \cdot \hat{s})$. This step effectively reverses the encryption masking process by removing the contribution of the secret key and the sender's ephemeral randomness, leaving only the original encoded message. Finally, the resulting polynomial is decoded to recover the original message $m$.

**Algorithm 4** Kyber Decryption

**Input:** Ciphertext $ct$, Secret Key $s_k$ **Output:** Message $m$

Parse $ct$ into $(\hat{u}, v)$
$s \leftarrow \text{Decode}(s_k)$
$\hat{s} \leftarrow \text{NTT}(s)$
$m' \leftarrow \text{Decode}(v - \text{InverseNTT}(\hat{u} \odot \hat{s}))$
**return** $m'$

To sign a message $m$, the drone first parses its Dilithium secret key dsk, which contains the secret vectors $s_1$, $s_2$, the decomposed component $t_0$, the matrix seed $\rho$, and a personalization value $K$. These components are essential for both security and reproducibility. The algorithm 5 begins by computing a message digest $\mu$ using SHAKE-256 over the concatenation of $K$ and $m$. This digest ensures that the signature is both message-dependent and cryptographically bound to the device. A uniformly random masking vector $y$ is sampled to protect the secret key during signing. The public matrix $\hat{A}$ is regenerated from the seed $\rho$, and a matrix–vector product is computed: $w = \hat{A} \cdot y$. This result is then compressed into its higher-order bits $w_1$, which are used — along with $\mu$ — to derive the challenge polynomial $c$ via a hash function. The core of the signature is computed as $z = y + c \cdot s_1$, which must remain within predefined norm bounds to ensure security. If the computed values do not satisfy these bounds (checked via rejection sampling), the process repeats with a new $y$. Once acceptable values

are produced, hint bits $h$ are generated to help the verifier reconstruct $w_1$ efficiently without leaking information. The final signature is constructed by encoding the tuple $(z, h, c)$, which is returned as sig.

**Algorithm 5** Dilithium Signature Generation at Drone

**Input:** Message $m$, Secret Key $dsk$ **Output:** Signature $sig$

Parse $dsk$ into $(s_1, s_2, t_0, \rho, K)$
$\mu \leftarrow$ SHAKE-256$(K \| m)$
**repeat**
  $y \leftarrow$ SampleUniform()
  $w \leftarrow \hat{A} \cdot y$
  $w_1 \leftarrow$ HighBits$(w)$
  $c \leftarrow$ H$(\mu \| w_1)$
  $z \leftarrow y + c \cdot s_1$
**until** NormBoundCheck$(z, w)$ passes
$h \leftarrow$ MakeHint$(w, w_1)$
$sig \leftarrow$ Encode$(z, h, c)$

The algorithm 6 describes the Dilithium signature verification. To verify a signature on a message $m$, the verifier receives the signature sig $= (z, h, c)$ and the public key dpk, which contains the matrix seed $\rho$ and the compressed public polynomial $t_1$. The algorithm 6 first checks whether the signature component $z$ satisfies the norm constraints defined by the Dilithium specification. If it fails this bound check, the signature is immediately rejected. Next, the matrix $\hat{A}$ is regenerated deterministically from the seed $\rho$, and the commitment value $w'$ is reconstructed as $w' = \hat{A} \cdot z - c \cdot t_1$. Since $w'$ is compressed during signature generation, the verifier uses the hint bits $h$ to recover its high-order components $w'_1$, approximating the original compressed value. The verifier then recomputes the message hash $\mu$ by hashing the personalization value $K$ concatenated with the message $m$, assuming $K$ is either included in the public key or securely derivable. Using $\mu$ and the reconstructed $w'_1$, the verifier derives a new challenge $c'$ using the same hash function as the signer. If the recomputed challenge $c'$ matches the challenge $c$ embedded in the signature, the signature is considered valid, and the algorithm returns "Accept". Otherwise, it returns "Reject".

**Algorithm 6** Dilithium Signature Verification at Drone

**Input:** Message $m$, Signature $sig = (z, h, c)$, Public Key $dpk$
**Output:** Accept or Reject

Parse $dpk$ into $(t_1, \rho)$
Parse $sig$ into $(z, h, c)$
**if** NormBoundCheck$(z)$ fails **then**
  **return** Reject
**end if**
$\hat{A} \leftarrow$ GenMatrix$(\rho)$
$w' \leftarrow \hat{A} \cdot z - c \cdot t_1$
$w'_1 \leftarrow$ UseHint$(w', h)$
$\mu \leftarrow$ SHAKE-256$(K \| m)$
$c' \leftarrow$ H$(\mu \| w'_1)$
**if** $c' = c$ **then**
  **return** Accept
**else**
  **return** Reject
**end if**

## 6. Security analysis

This section presents the security analysis of the proposed quantum-secure secure communication protocol that integrates SRAM PUF with NIST-standard lattice-based schemes: Kyber (for encryption) and Dilithium (for digital signatures). The analysis relies on the PUF's physical assumptions and the hardness assumptions of the M-LWE/SIS problems underlying Kyber and Dilithium.

### 6.1. Formal security proof

We evaluate the protocol against quantum adversaries under standard security definitions. This section presents the formal security proof of the proposed quantum-secure communication protocol using Burrows–Abadi–Needham (BAN) logic and Scyther to analyze and ensure the overall security of the protocol.

#### 6.1.1. Security proof using BAN logic

To formally validate the authentication and trust guarantees of the proposed protocol between drones and GCS, we apply the BAN logic. BAN logic is a formal method used to analyze the correctness of authentication protocols by modeling participating entities beliefs and trust relationships. The logic uses a set of inference rules to reason about what each principal believes after receiving and interpreting messages within a protocol.

This approach abstracts actual protocol messages into idealized forms and applies logical transformations to deduce beliefs about message authenticity, freshness, ownership of secrets, and key integrity. In this analysis, we aim to prove that after the protocol execution:

- The drones ($D1$, $D2$) and the GCS agree on shared keys.
- All transmitted keys are fresh and uncompromised.
- Each entity correctly believes in the authenticity and validity of the received keys and identity assertions.

To facilitate this, we define the following:

- The protocol goals to be proven (G1–G4).
- The actual message flows between $D1$, $D2$, and GCS.
- The idealized messages used for logical deduction.
- The assumptions (A1–A14) about freshness, jurisdiction, and initial beliefs.

Using the defined BAN logic rules (Table 3) and notations (Table 2), we derive step-by-step proofs (S1–S25) as given below:

The following objectives must be attained by the proposed protocol:

**Goal 1:** $D2 \mid\equiv (D2 \overset{ChD1, ResD1, ResD2}{\Longleftrightarrow} GCS)$

**Goal 2:** $GCS \mid\equiv (D2 \overset{ChD1, ResD1, ResD2}{\Longleftrightarrow} GCS)$

**Goal 3:** $D1 \mid\equiv (D1 \overset{SK, ResD1}{\Longleftrightarrow} D2)$

**Goal 4:** $D2 \mid\equiv (D1 \overset{SK, ResD1}{\Longleftrightarrow} D2)$

The messages of the protocol are:

Message 1: D1,T1
Message 2: m2
Message 3: m3
Message 4: m4
Message 5: m5,PK1
Message 6: m6
Message 7: m7
Message 8: m8, PK2
Message 9: m9

The idealized messages are as follows:

Message 1: D1→ D2: T1, D1.
Message 2: D2 → GCS: $\{\langle D1, T1, D2ID\rangle_{PK_{GCS}}\}$.
Message 3: GCS → D2: $\{\langle ChD2\rangle_{PK_{D2}}\}$.
Message 4: D2 → GCS: $\{\langle ResD2\rangle_{PK_{GCS}}\}$.
Message 5: GCS → D2: $\{PKD1, \langle H(PK_{D1}), D1ID, TS, ChD1, ResD1\rangle_{PK_{D2}}\}$.
Message 6: D2 → D1: $\{\langle ChD1, SK\rangle_{PK_{D1}}\}$.
Message 7: D1 → D2: $\{\langle ResD1, DPK_{D1}\rangle_{PK_{D2}}\}$.
Message 8: D2 → D1: $\{PK_{D2}, \langle AckD2, H(PK_{D2}), D2ID\rangle_{PK_{D1}}\}$.
Message 9: D1 → D2: $\{\langle SignDSK_{D1}(H(M), M)\rangle_{PK_{D2}}\}$.

The following are the assumptions of the proposed scheme:

A1: $D2 \mid\equiv \#(T1)$; A2: $D2 \mid\equiv \#(TS)$; A3: $GCS \mid\equiv D2 \Rightarrow D1, T1, D2ID$; A4: $D2 \mid\equiv GCS \Rightarrow ChD2$; A5: $GCS \mid\equiv D2 \Rightarrow ResD2$; A6: $D2 \mid\equiv GCS \Rightarrow$

**Table 3**
Selected BAN logic inference rules.

| | | |
|---|---|---|
| Message Meaning Rule: | If principal $R$ believes that $n$ is a shared secret between $R$ and $Q$, and $R$ receives a message $S$ bundled with $n$, then $R$ concludes that $Q$ must have previously said $S$. | $\frac{R\mid\equiv Q\overset{n}{\leftrightharpoons}R,\quad R\lhd\langle S\rangle_n}{R\mid\equiv Q\mid\sim S}$ |
| Nonce Verification Rule: | If $R$ trusts that message $S$ is fresh and also believes that $Q$ has said $S$ at some point, then $R$ can infer that $Q$ currently believes $S$. | $\frac{R\mid\equiv\#(S),\quad R\mid\equiv Q\mid\sim S}{R\mid\equiv Q\mid\equiv S}$ |
| Jurisdiction Rule: | If $R$ is confident that $Q$ has authority over the statement $S$ and that $Q$ believes $S$, then $R$ is justified in believing $S$ as well. | $\frac{R\mid\equiv Q\Rightarrow S,\quad R\mid\equiv Q\mid\equiv S}{R\mid\equiv S}$ |
| Freshness Rule: | If $R$ believes that a part of a message, say $S$, is fresh, then $R$ can also trust that the entire message composed of $S$ and another part $V$ is fresh. | $\frac{R\mid\equiv\#(S)}{R\mid\equiv\#(S,V)}$ |
| Belief Extraction Rule: | If $R$ believes that $Q$ either currently believes or previously said $(S,V)$, then $R$ may also conclude that $Q$ believes or once said $S$ alone. Similarly, if $R$ believes $(S,V)$, then $R$ also believes $S$. | $\frac{R\mid\equiv Q\mid\equiv(S,V)}{R\mid\equiv Q\mid\equiv S}$, $\frac{R\mid\equiv Q\mid\sim(S,V)}{R\mid\equiv Q\mid\sim S}$, $\frac{R\mid\equiv(S,V)}{R\mid\equiv S}$ |

$H(PK_{D1}), D1ID, TS, ChD1, ResD1$; A7: $D1 \mid\equiv D2 \Rightarrow ChD1, SK$; A8: $D2 \mid\equiv D1 \Rightarrow ResD1, DPK_{D1}$; A9: $D1 \mid\equiv D2 \Rightarrow AckD2, H(OK_{D2}), D2ID$; A10: $D2 \mid\equiv D1 \Rightarrow SignDSK_{D1}(H(M), M)$; A11: $D1 \mid\equiv T1, ResD1$; A12: $GCS \mid\equiv TS$; A13: $D2 \mid\equiv SK, D2ID, ResD2$; A14: $GCS \mid\equiv ChD2, ChD1, ResD1$;

By considering the aforementioned assumptions and merging Message 2 with Message 4, we derive the below result:

- S1: $GCS \lhd \{\langle T1, D1, D2ID\rangle_{PK_{GCS}}\}$ and $GCS \lhd \{\langle ResD2\rangle_{PK_{GCS}}\}$.
- S2: According to the belief rule, $GCS \lhd \langle T1, D1, D2ID\rangle_{PK_{GCS}}\}$ and $GCS \lhd \langle ResD2\rangle_{PK_{GCS}}\}$
- S3: Applying message meaning rule and A6, GCS $\mid\equiv D2 \mid\sim (T1, D1, D2ID)$ and $GCS \mid\equiv D2 \mid\sim (ResD2)$.
- S4: Applying A1, A2 and the freshness rule, $GCS \mid\equiv \#(T1, D1, D2ID)$ and $GCS \mid\equiv \#(ResD2)$.
- S5: Applying nonce verification rule, $GCS \mid\equiv D2 \mid\equiv (T1, D1, D2ID)$ and $GCS \mid\equiv D2 \mid\equiv (ResD2)$.
- S6: Applying A4, A5 and the jurisdiction rule, GCS $\mid\equiv (T1, D1, D2ID)$ and $GCS \mid\equiv (ResD2)$.
- S7: Applying A14 and belief rule, GCS $\mid\equiv T1, GCS \mid\equiv D1, GCS \mid\equiv D2ID$, and $GCS \mid\equiv ResD2$, Goal 2 is satisfied, $GCS \mid\equiv (D2 \xleftrightarrow{ChD1, ResD1, ResD2} GCS)$

By combining m3 and m5, we deduce,

- S8: Applying belief rule, $D2 \lhd (ChD2, PK_{D1}, \langle H(PK_{D1}), D1ID, TS, ChD1, ResD1\rangle_{PK_{D2}})$
- S9: Applying A14 and the message meaning rule, $D2 \mid\equiv GCS \mid\sim (ChD2, PK_{D1}, H(PK_{D1}), D1ID, TS, ChD1, ResD1)$.
- S10: Applying the freshness rule and A3, $D2 \mid\equiv \#(ChD2, PK_{D1}, H(PK_{D1}), D1ID, TS, ChD1, ResD1)$.
- S11: Applying the nonce verification rule, $D2 \mid\equiv GCS \mid\equiv (ChD2, PK_{D1}, H(PK_{D1}), D1ID, TS, ChD1, ResD1)$.
- S12: Applying A4, jurisdiction rule, $D2 \mid\equiv (ChD2, PK_{D1}, H(PK_{D1}), D1ID, TS, ChD1, ResD1)$.
- S13: Applying A13 and belief rule, D2 $\mid\equiv ChD2, D2 \mid\equiv PK_{D1}, D2 \mid\equiv H(PK_{D1}), D2 \mid\equiv D1ID, TS, D2 \mid\equiv ChD1, ResD1$, Goal 1 is satisfied, $D2 \mid\equiv (D2 \xleftrightarrow{ChD1, ResD1, ResD2} GCS)$

By combining m6 and m8, we deduce,

- S14: Applying belief rule, $D1 \lhd ChD1, SK, PK_{D2}, \langle AckD2, H(PK_{D2}), D2ID\rangle_{PK_{D2}}$
- S15: Applying A14 and the message meaning rule, $D1 \mid\equiv D2 \mid\sim (ChD1, SK, PK_{D2}, AckD2, H(PK_{D2}), D2ID)$.
- S16: Applying the freshness rule and A3, $D1 \mid\equiv \#(ChD1, SK, PK_{D2}, AckD2, H(PK_{D2}), D2ID)$.
- S17: Applying the nonce verification rule, $D1 \mid\equiv D2 \mid\equiv (ChD1, SK, PK_{D2}, AckD2, H(PK_{D2}), D2ID)$.
- S18: Applying A4, jurisdiction rule, $D1 \mid\equiv (ChD1, SK, PK_{D2}, AckD2, H(PK_{D2}), D2ID)$.
- S19: Applying belief rule, D1 $\mid\equiv ChD1, D1 \mid\equiv SK, D1 \mid\equiv H(PK_{D2}), D1 \mid\equiv AckD2, D1 \mid\equiv PK_{D2}, D1 \mid\equiv D2ID$. Hence from A11 and S19, Goal 3 is satisfied, $D1 \mid\equiv (D1 \xleftrightarrow{SK, ResD1} D2)$.

For m7, we deduce,

- S20: Applying belief rule, $D2 \lhd \langle ResD1, DPK_{D1}\rangle_{SK}$
- S21: Applying A14 and the message meaning rule, $D2 \mid\equiv D1 \mid\sim (ResD1, DPK_{D1})$.
- S22: Applying the freshness rule and A3, $D2 \mid\equiv \#(ResD1, DPK_{D1})$.
- S23: Applying the nonce verification rule, $D2 \mid\equiv D1 \mid\equiv (ResD1, DPK_{D1})$.
- S24: Applying jurisdiction rule, $D2 \mid\equiv (ResD1, DPK_{D1})$.
- S25: Applying A13 and belief rule, D2 $\mid\equiv ResD1, D2 \mid\equiv DPK_{D1}$, Goal 4 is achieved, $D2 \mid\equiv (D1 \xleftrightarrow{SK, ResD1} D2)$.

#### 6.1.2. Security proof using scyther

Scyther is an open-source automated tool designed for the formal verification of cryptographic protocols. Developed to analyze security properties under the Dolev–Yao adversary model, it allows researchers to define communication sequences, cryptographic operations, and desired security goals using its dedicated modeling language, SPDL (Scyther Protocol Description Language). The tool systematically explores all possible execution traces and identifies potential vulnerabilities, such as MitM, replay, or impersonation attacks. It supports both symbolic reasoning and the generation of attack traces, providing a comprehensive framework to evaluate authentication and secrecy claims. Scyther operates under the perfect cryptography assumption, where cryptographic primitives such as encryption and hashing are treated as black boxes, impervious to brute-force or algebraic attacks. This allows Scyther to focus exclusively on the logical flaws within the protocol flow, enabling efficient verification of even complex protocols with multiple roles and sessions.

Scyther uses claims to express the intended security goals of a protocol. These claims are inserted within each role in the SPDL specification and serve as assertions that must hold true for the protocol to be considered secure. The most commonly used claim types include:

- Secret: The claim Secret is used to verify that a message m remains confidential from all potential attackers throughout the execution of the protocol. This claim helps confirm that no part

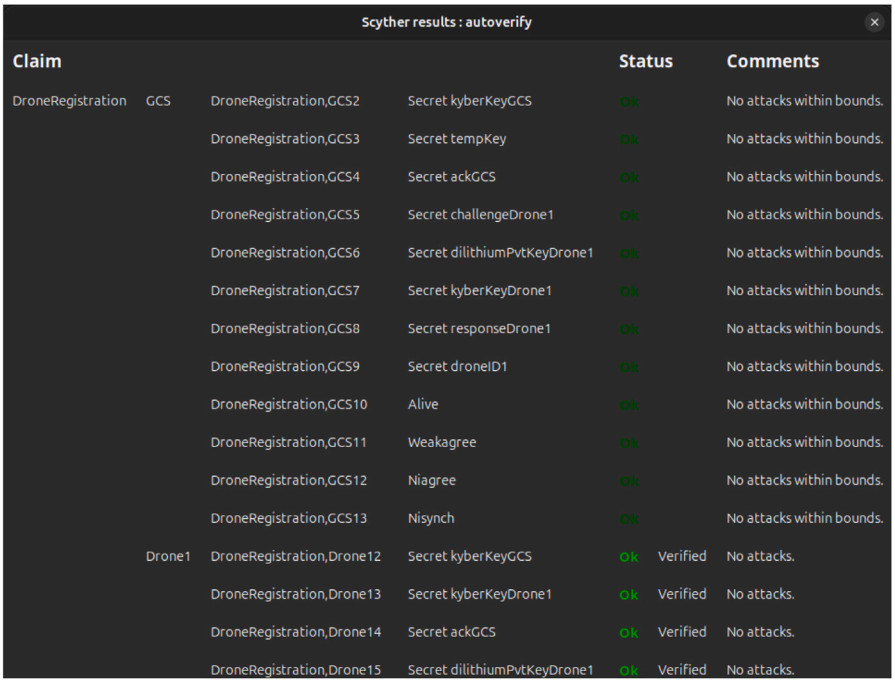



**Fig. 3.** Scyther output for Drone1 registration and secrecy verification.

of the communication unintentionally leaks sensitive information. It is particularly important when evaluating the security of keys, credentials, or session-specific values.

- Alive: This claim ensures that a particular role has reached a specific point in the protocol, indicating that the party is actively participating and has not been terminated prematurely. It helps detect denial-of-service conditions or instances where a protocol execution halts unexpectedly. This claim does not guarantee successful interaction with other roles but simply that the execution proceeded to the intended point.
- Weakagree: The claim Weakagree is used to check that there is some level of agreement between parties in a protocol run. It ensures that both roles engage in the protocol and use the same values, but without verifying the uniqueness or ordering of sessions. This is often used in simple authentication protocols where basic mutual trust is sufficient.
- Niagree: The Niagree or non-injective agreement claim checks that the values received during a protocol run match what the peer role claims to have sent. It ensures that both participants agree on the data exchanged, although it does not require each run to be uniquely matched. This is useful in protocols where concurrency is allowed, and verifying data integrity between parties is the goal.
- Nisynch: This claim checks that the entire message flow and state transitions of the protocol are synchronized between two parties. It ensures that all messages are received in the expected order without replay or injection attacks. This form of synchronization validation is critical for strong authentication and session integrity.
- Commit: The claims Commit and Running are used together to confirm that role X believes it has completed a protocol with role Y, and vice versa. These claims help validate that both parties were engaged in the same session with matching input and output parameters. Their successful validation prevents impersonation and confirms the integrity of the session's context.
- Reachable: The claim Reachable in Scyther checks whether a specific point in a particular role execution is accessible under the defined protocol and adversary model. It verifies that the protocol run can reach that line during some execution trace. This often confirms that the protocol is not deadlocked or overly restrictive and can progress normally under valid conditions.

The proposed drone communication protocol has been formally modeled and verified using the Scyther tool to ensure its resilience against a wide range of symbolic adversaries. The protocol encompasses drone registration with the GCS, GCS-to-drone communication, and secure inter-drone communication. Each phase was described using the SPDL, with explicit claims for secrecy, agreement, authentication, and freshness.

In the registration and key exchange phase, each drone shares its unique identifier with the GCS and receives cryptographic materials required for establishing a secure communication session. The GCS sends a challenge to each drone, and the drone responds to it. As illustrated in Figs. 3 and 4, Scyther validated secrecy claims for critical parameters such as `kyberKeyGCS`, `kyberKeyDrone1`, `dilithiumPvtKeyDrone1`, `ackGCS`, and `responseDrone1`. All secrecy claims passed successfully, denoted by "`ok`" and "`Verified`" statuses, indicating that no sensitive value was exposed or leaked during symbolic execution.

The next phase involved verifying message agreement properties. This was achieved through the claims `Alive`, `Weakagree`, and `Niagree`, ensuring that the parties are live, in agreement on exchanged values, and mutually committed to the communication. These properties were validated in both the GCS and drone roles. The `Commit` claims involving exchanged identifiers, challenges, and acknowledgments were also validated without any violations.

After successful mutual authentication and key exchange, inter-drone communication is implicitly verified through transitive trust. Since both drones possess each other's validated and secret keys, they can engage in secure peer-to-peer communication using the authenticated and encrypted channels established during registration. The confidentiality of these keys is assured by the previously validated secrecy claims, while freshness is maintained by the validated challenge–response mechanism.

Although Scyther models communication between two parties at a time, the collective verification results ensure end-to-end security of

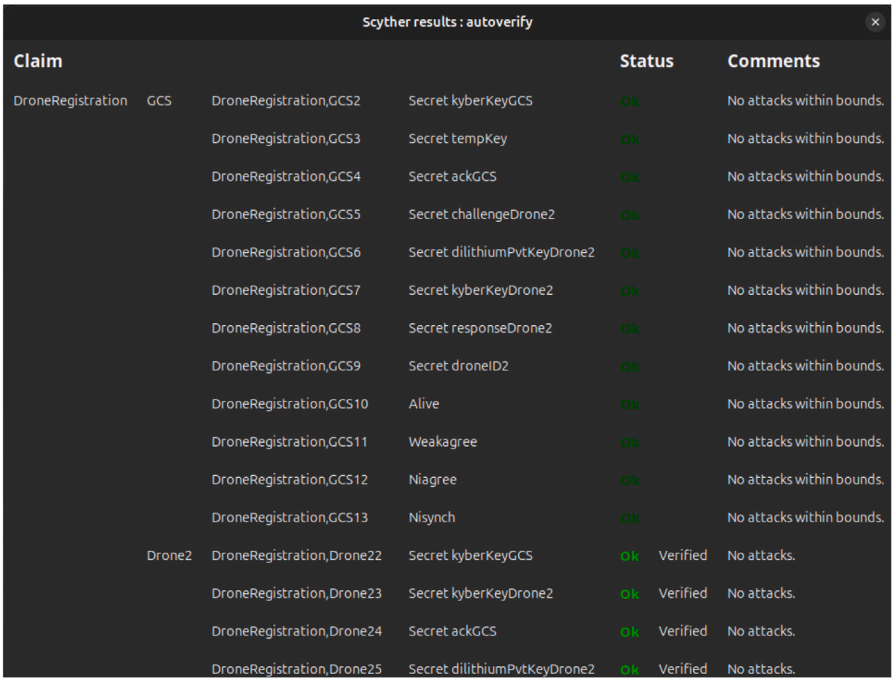



**Fig. 4.** Scyther output for Drone2 registration and secrecy verification.

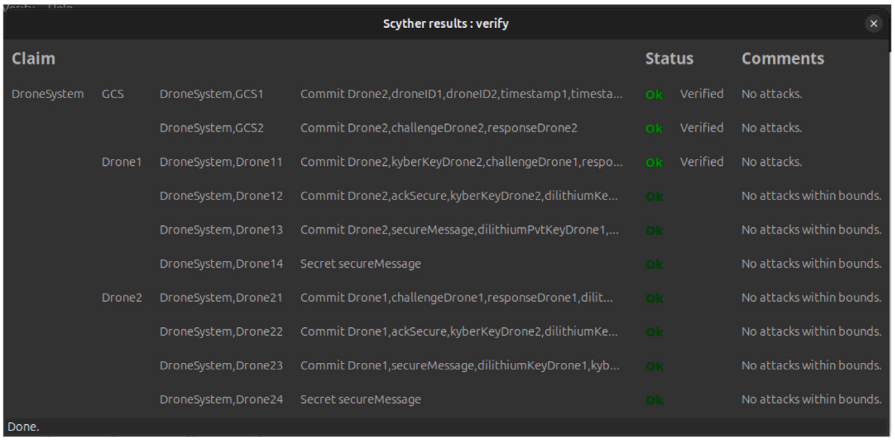



**Fig. 5.** Combined verification results for all roles in DroneSystem.

the overall system. Since the GCS intermediates all initial exchanges and distributes verified cryptographic credentials, the trust extends to Drone1–Drone2 communication securely. This is supported by the outputs shown in Fig. 5, which confirm the lack of attacks in the combined verification phase involving all three roles.

The Scyther analysis confirms that the modeled protocol is secure against classical symbolic threats. The validation of all secrecy, agreement, and commit claims across all roles indicates robustness in the protocol's design. The use of Kyber and Dilithium post-quantum primitives further reinforces its security posture for deployment in real-world UAV networks under future quantum-threat conditions. The complete SPDL specification of the proposed protocol, detailing the roles, message exchanges, and formal claims used in the verification process, is provided in Appendix for reference.

### 6.2. Informal security proof

To evaluate the proposed protocol's robustness, we present an informal security analysis covering confidentiality, authenticity, and resistance against a range of classical and quantum attack vectors (Ali et al., 2025). This will provide an outline of how integrating Kyber and Dilithium, reinforced by PUF-based randomness, provides strong cryptographic guarantees. Table 4 presents a comparative analysis of the proposed protocol's resistance to various security attacks in comparison with existing works. The following analysis substantiates the protocol's resilience in resource-constrained and adversarial environments.

#### 6.2.1. Key confidentiality and IND-CCA2 security

The key encapsulation mechanism in Kyber ensures IND-CCA2 security (Bos et al., 2018), given the use of a deterministic PRF with a seed derived from PUF-based randomness (Algorithm 1). The derived key pair $(sk, pk)$ is used to generate session keys securely. Using the NTT transformation, polynomial sampling via CBD, and matrix–vector arithmetic over $\mathbb{Z}_q$ ensures resistance against classical and quantum attacks.

#### 6.2.2. Message authenticity and unforgeability

Digital signatures are generated using Dilithium (Algorithm 5), where signatures are bound to both the message hash $\mu = H(K \parallel m)$

**Table 4**
Comparison of attack resistance features with existing works.

| Security properties | Kumari et al. (2022) | Ye et al. (2024) | Mustafa et al. (2020) | Choi and Lee (2024) | Mansoor et al. (2024a) | Zhang et al. (2020) | Tian et al. (2019) | Pu et al. (2022) | Yu et al. (2022) | Proposed protocol |
|---|---|---|---|---|---|---|---|---|---|---|
| Key Confidentiality | ✓ | ✗ | ✓ | ✓ | ✓ | ✓ | ✗ | ✓ | ✓ | ✓ |
| Message Authenticity and Unforgeability | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Resistance to Replay Attacks | ✓ | ✗ | ✓ | ✓ | ✗ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Mutual Authentication | ✓ | ✗ | ✓ | ✓ | ✓ | ✓ | ✗ | ✓ | ✓ | ✓ |
| Resistance to Key Forgery and Substitution | ✓ | ✗ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Resistance to Man-in-the-Middle Attacks | ✓ | ✗ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| De-synchronization and Deserialization Resistance | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✗ | ✓ |
| Resistance to Signature Replay or Injection | ✓ | ✗ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| Resistance to Eavesdropping and Ciphertext Malleability | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

and the secret key components. The verification logic (Algorithm 6) confirms the validity of the signature without revealing any information about the private key, thus ensuring UF-CMA security based on the hardness of the SIS problem (Jackson et al., 2024).

#### 6.2.3. Resistance to replay attacks

All messages exchanged between drones and between a drone and the GCS are encrypted using the Kyber algorithm, which employs unique randomness $r$ generated via SHAKE-128 from fresh input, ensuring semantic security. Replay attacks are mitigated since Kyber ciphertexts are non-deterministic, meaning the same message will always produce a different ciphertext. The Dilithium digital signatures are tightly bound to both the message content and the unique randomness through hash chaining and the challenge–response mechanism of the PUF. This design ensures that any attempt to reuse or replay previously transmitted data will be detected and rejected by the proposed system.

#### 6.2.4. Mutual authentication

The protocol supports mutual authentication using Dilithium signatures derived from PUF-based keys. Since PUF responses are unique per device and per challenge, only the legitimate drone possessing the correct PUF can regenerate the corresponding private key and sign messages correctly. Verification succeeds only if both the message integrity and PUF-bound identity are validated.

#### 6.2.5. Resistance to key forgery and substitution

Due to the use of PUF-derived seeds, it is computationally infeasible to replicate a device's key material without physical access to the device. An attacker cannot regenerate $\rho$, $\sigma$, or $K$ without the exact SRAM behavior. Thus, key forgery or substitution by an adversary is rendered infeasible, even with access to quantum computational capabilities.

#### 6.2.6. Resistance to Man-in-the-Middle (MitM) attacks

In the proposed protocol, Kyber is employed for encrypted key exchange and Dilithium for authenticated signing between drones (D1 and D2) and between each drone (D1) and the GCS. This dual use of encryption and digital signatures ensures that MitM attacks are ineffective unless both cryptographic mechanisms are simultaneously compromised. Any attempt by an attacker to modify a transmitted message will alter the ciphertext, leading to failed decryption and authentication at the receiver's end. As the security of Kyber and Dilithium relies on M-LWE and SIS problems respectively, their joint compromise remains computationally infeasible under quantum adversaries.

#### 6.2.7. De-synchronization and deserialization resistance

The inclusion of consistent PUF-based entropy and standardized polynomial transformations in the key generation process ensures that, even under interrupted transmissions or protocol resets, re-initialization with the same challenge results in the same key pair. This enables robust, self-healing synchronization across sessions.

#### 6.2.8. Resistance to signature replay or injection

After D1 and D2 are mutually authenticated, the subsequent communication is secured using Dilithium digital signature scheme. Algorithm 5 enforces a tight bound on signature norms and randomness ($z$) using rejection sampling, making it difficult for an adversary to forge a valid signature through replay. The signature encoding also includes a hint ($h$), which is linked to the secret key's decomposition, further enhancing resistance to signature forgery.

#### 6.2.9. Resistance to eavesdropping and ciphertext malleability

In the proposed protocol, all communications are secured using Kyber encryption (Algorithm 3) that employs randomized CBD-based vector sampling and matrix multiplication, producing ciphertexts resistant to malleability and decryption oracle attacks. Even if an attacker attempts to eavesdrop on the communication and intercepts multiple ciphertexts, the randomized nature of Kyber ensures that each encryption instance produces a unique ciphertext for the same plaintext. Hence the intercepted data provides no meaningful information about the original message, owing to the semantic security guaranteed by Kyber's IND-CCA2 property. This prevents eavesdropping and protects the confidentiality of all transmitted data in our system.

## 7. Implementation details

### 7.1. PUF-enabled key generation for Kyber and Dilithium

In the proposed system, SRAM PUF are utilized to generate secure, device-specific keys for both Kyber encryption and Dilithium signature schemes. SRAM PUF exploit the intrinsic manufacturing variations in the memory cells of SRAM to produce a unique and reproducible pattern upon every power-up. These random values that appear when the device is powered on are used to create cryptographic seeds. This helps each device have its own unique identity based on its hardware, without the need to store any secret keys in memory.

In our implementation, the SRAM PUF is embedded via Arduino into the drone's hardware platform, allowing seamless integration without significant overhead. When a drone initiates communication, it uses the PUF output to dynamically derive the keys for Kyber and Dilithium. This approach eliminates the risks associated with storing static keys,

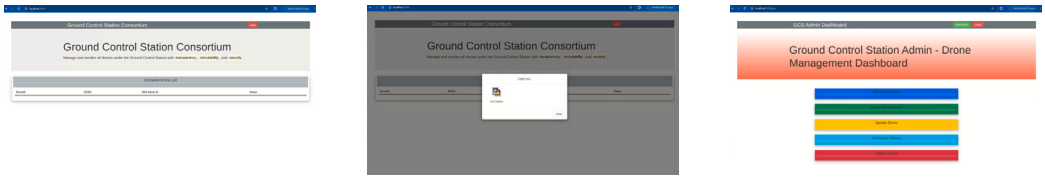



**Fig. 6.** Graphical User Interface developed for testing the drone credentials stored in the hyperledger fabric blockchain by GCS Admin.

significantly reducing the system's vulnerability to physical tampering and key extraction attacks.

One of the inherent strengths of SRAM PUF is their ability to produce highly unique and unpredictable outputs, which makes them good choice for authentication and secure key generation in resource-constrained IoD environments. However, environmental factors such as temperature and supply voltage variations can influence the stability of the PUF output. To mitigate this, robust error correction techniques and helper data algorithms are employed to ensure consistent key reproduction without compromising security.

### 7.2. Kyber

The cryptographic operations, including encryption and decryption using Kyber, were tested on both Raspberry Pi 4 (Quad-core Cortex-A72, ARM v8, 64-bit, 1.5 GHz) and Raspberry Pi 5 (Quad-core Cortex-A76, ARM v8.2-A, 64-bit, 2.4 GHz). These evaluations were performed using Python on a 64-bit Raspberry Pi OS built on Debian Bullseye. Also, performance testing was carried out on a desktop system featuring an Intel Core i5-11500 CPU at 2.70 GHz with 16 GB of RAM, running a 64-bit Linux environment. In this setup, the secret keys are generated by extracting SRAM PUF responses from an Arduino board via a serial port. This unique hardware fingerprint is then used as the seed input for the Kyber512 key generation algorithm, ensuring that the keys are generated securely without the need for permanent storage on the device.

### 7.3. Dilithium

To evaluate the efficiency of digital signature operations, specifically Dilithium signing and verification, tests were conducted on both Raspberry Pi 4 (Quad-core Cortex-A72, ARM v8, 64-bit, 1.5.GHz) and Raspberry Pi 5 (Quad-core Cortex-A76, ARM v8.2-A, 64-bit, 2.4.GHz). These tests were performed using Python on a 64-bit Raspberry Pi OS based on Debian Bullseye. Further evaluations were carried out on a system with an Intel Core i5-11500 processor clocked at 2.70.GHz, 16.GB of RAM, and running a 64-bit Linux environment. In the proposed setup, an SRAM PUF on an Arduino device was used to generate a unique seed, which was accessed via the serial port. This seed was then used for key generation in the Dilithium algorithm, allowing each device to create a cryptographic identity that is bound to its hardware, while avoiding the need to store static keys in memory.

### 7.4. Hyperledger fabric blockchain

The GCS is configured with Hyperledger Fabric version 2.2 on a PC running Ubuntu 22.04. The blockchain network is configured using Minifabric and the IBM Blockchain Platform extension, while the RAFT consensus algorithm is adopted to ensure reliable and synchronized operation across all participating nodes. In the proposed framework, the GCS acts as a blockchain network administrator and manages a permissioned private blockchain. GCS is modeled as an organization within the network, with a peer node and a certificate authority (CA) responsible for identity management. In the enrollment phase, the credentials of the drone, including its ID, PUF challenge–response pairs, Kyber public key, and Dilithium public key, are securely stored in the blockchain ledger. To protect against quantum threats, all sensitive credential data is encrypted using Kyber encryption before storage on the blockchain, ensuring confidentiality even in adversarial environments.

Chaincodes are developed using JavaScript, which define the smart contract logic and are deployed on the Fabric network to automate credential storage. It consists of one drone contract, which provides functions related to drone operations that can be executed by the GCS administrator. The implemented drone contract provides various functions including registering a new drone, retrieving drone details, modifying existing drone details, viewing the full transaction history of a specific drone, and deleting a drone from the world state without erasing its record from the blockchain. The functionality was validated through interaction with a graphical user interface, which is shown in Fig. 6.

## 8. Performance analysis and results

This section analyzes the performance of our proposed quantum-secure drone-to-drone authentication and communication protocol developed for the IoD. The evaluation focuses on performance metrics, including computational cost, communication overhead, CPU utilization, and memory usage during the authentication and message communication phases. The experiments were conducted using Raspberry Pi 4 and Raspberry Pi 5 boards to emulate drones, while a desktop system with an Intel Core i5-11500 CPU running at 2.70 GHz, 16 GB RAM, and Linux OS acted as the GCS to implement and test the Hyperledger Fabric framework. The performance of Hyperledger Fabric blockchain is evaluated using the Caliper tool by measuring latency, throughput, CPU usage, memory consumption, and network traffic under different drone configurations during ledger read and write operations.

Along with the experimental evaluation, we have added a feature-based comparison shown in Table 5, emphasizing the distinct advantages of the proposed protocol over existing state-of-the-art methods. Primary comparison dimensions include domain applicability, quantum security, physical security, drone-to-drone authentication, blockchain-based credential management, deployment on resource-constrained hardware, secure communication, and formal security validation. The proposed protocol stands out by addressing all these aspects, making it well-suited for secure IoD environments.

### 8.1. Analysis of computation cost in the proposed protocol

Computational cost refers to the total time taken to execute a protocol, which varies based on the chosen hardware platform and the complexity of the underlying computational steps. The computations in the proposed drone-to-drone authentication and communication protocol mainly include key generation, hash operations, PUF operations, Kyber encryption, decryption, Dilithium signing and verification. Let $T_{kg}$, $T_h$, $T_{puf}$, $T_{ke}$, $T_{kd}$, $T_{ds}$, and $T_{dv}$ denote the time for key generation, hash operations, PUF response, Kyber encryption, Kyber decryption, Dilithium signature generation, and Dilithium signature verification.

**Table 5**
Comparison of the proposed protocol with existing works.

| Authors | Domain | Quantum secure | Physical security | Drone-to-drone authentication | Blockchain credential management | Tested on low power hardware | Secure communication | Formal validation |
|---|---|---|---|---|---|---|---|---|
| Kumari et al. (2022) | IoT | ✓ | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ |
| Ye et al. (2024) | IoT | ✓ | ✗ | ✗ | ✗ | ✓ | ✓ | ✗ |
| Mustafa et al. (2020) | IoT | ✓ | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ |
| Choi and Lee (2024) | IoT | ✓ | ✗ | ✗ | ✗ | ✓ | ✓ | ✗ |
| Mansoor et al. (2024a) | IoT | ✓ | ✗ | ✗ | ✗ | ✓ | ✓ | ✗ |
| Zhang et al. (2020) | IoD | ✗ | ✗ | ✗ | ✗ | ✓ | ✓ | ✓ |
| Tian et al. (2019) | IoD | ✗ | ✗ | ✓ | ✗ | ✓ | ✓ | ✗ |
| Pu et al. (2022) | IoD | ✗ | ✓ | ✓ | ✗ | ✓ | ✓ | ✓ |
| Yu et al. (2022) | IoD | ✗ | ✓ | ✗ | ✗ | ✓ | ✓ | ✓ |
| **Proposed Protocol** | IoD | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

**Table 6**
Time taken for each individual operation for authentication using RPi4 as Drone and i5 PC as GCS.

| Operation | Execution time (s) | Processor utilization (%) | Memory utilization (MB) | Estimated power (W) | Total energy (J) |
|---|---|---|---|---|---|
| $T_{kg}$ | 0.0337 | 2.499 | 0.7049 | 2.822 | 0.0952 |
| $T_{puf}$ | 2.053 | 3.00 | 0.417 | 2.847 | 5.8254 |
| $T_h$ | 0.0005265 | 2.542 | 0.234 | 2.825 | 0.0015 |
| $T_{kea}$ | 0.0131457 | 2.38 | 0.191 | 2.817 | 0.0370 |
| $T_{kda}$ | 0.05007 | 4.6 | 0.312 | 2.925 | 0.1465 |

**Table 7**
Time taken for each individual operation for secure communication using RPi4 as Drone and i5 PC as GCS.

| Operation | Execution time (s) | Processor utilization (%) | Memory utilization (MB) | Estimated power (W) | Total energy (J) |
|---|---|---|---|---|---|
| $T_{kec}$ | 0.028 | 4.78 | – | 2.934 | 0.0822 |
| $T_{kdc}$ | 0.022 | 7.00 | – | 3.043 | 0.0669 |
| $T_{ds}$ | 0.08 | 2.02 | 0.246 | 2.799 | 0.2239 |
| $T_{dv}$ | 0.0654 | 5.47 | – | 2.968 | 0.1942 |

The cryptographic operations were tested separately on an Intel Core i5 desktop PC, Raspberry Pi 4, and Raspberry Pi 5 using the Lattice cryptographic library (IBM, 2022), with the Raspberry Pi boards representing drones. The corresponding results are detailed in Tables 6, 7, 8, and 9. For Raspberry Pi 4, the time taken to complete the authentication phase of the protocol is 0.2 s without considering the PUF data reading delay over the serial port, and 2.26 s when that delay is included. The secure communication step takes 0.19 s. On Raspberry Pi 5, the authentication step takes 0.08 s excluding serial reading time, and 2.15 s including it. The secure communication phase completes in 0.12 s. The reason for considering serial port delay is that in our setup, the SRAM PUF on the Arduino is connected to the Raspberry Pi through a serial interface. This delay could be minimized if the PUF values were directly accessed from the same device's internal SRAM.

Table 10 and Fig. 7 present the comparison of computation time between the proposed protocol and state-of-the-art PQC-based authentication schemes previously introduced by Kumari et al. (2022), Mansoor et al. (2024a), Ye et al. (2024), Mustafa et al. (2020), and Choi and Lee (2024). The proposed approach demonstrates a significant improvement in execution efficiency, achieving an authentication time of only 0.08 s on the Raspberry Pi 5. This reduced computation time, combined with low processor utilization and energy consumption, highlights the suitability of the proposed protocol in drone systems where fast processing and efficient resource usage are required.

Table 11 and Fig. 8 present a comparison of computation time with Kumari et al. (2022), Mansoor et al. (2024a), Ye et al. (2024), Mustafa et al. (2020), and Choi and Lee (2024). Among all, the proposed protocol achieves the lowest communication time of about 0.12 s when tested on Raspberry Pi 5. This efficient performance, along with moderate processor utilization and energy efficiency, demonstrates that the proposed scheme enables fast and reliable communication, making it well-suited for drone systems where quick and efficient communication is essential due to limited resources.

### 8.2. Analysis of communication and storage cost in the proposed protocol

Communication cost analysis looks at how much data is used and what resources are needed when sending information between systems. In our case, Kyber generates a ciphertext of 768 bytes and Dilithium produces a signature of 2420 bytes, and there are nine messages exchanged among the two drones and the GCS for both authentication and secure communication. Although our protocol uses slightly more communication data compared to some existing works during both the authentication (Kumari et al., 2022; Mansoor et al., 2024a; Ye et al., 2024; Mustafa et al., 2020; Choi and Lee, 2024) and communication phases (Kumari et al., 2022; Mansoor et al., 2024a; Ye et al., 2024; Choi and Lee, 2024), this increase is minor, especially when 5G technology is available. With the high speed and large bandwidth that 5G offers (Shahzadi et al., 2021), this extra data usage becomes less significant. The slight rise in communication cost is reasonable, considering the protocol performs faster, is more secure, and can protect against future quantum attacks.

The storage cost on the drone refers to the total amount of data it needs to keep in its memory. In our proposed method, this storage is limited to 805 bytes. This includes 800 bytes for storing the Kyber public key of the GCS, 4 bytes for a timestamp, and two 4 bit values, one representing the drone's ID and the other serving as a challenge to generate a 32 bit seed from the PUF. This seed is used to produce both Kyber and Dilithium keys. Importantly, none of the stored data is sensitive. Since no secret information is saved in the drone, even if an attacker gains access to its memory, there is nothing confidential to steal. This approach improves the protocol's overall security and makes it more resilient against potential breaches.

Even though our protocol has a slightly higher communication cost, the actual computation time is very low, which proves that it performs efficiently. The communication cost is also acceptable given that 5G networks provide high bandwidth. Also, the storage space used on the

**Table 8**
Time taken for each individual operation for authentication using RPi5 as Drone and i5 PC as GCS.

| Operation | Execution time (s) | Processor utilization (%) | Memory utilization (MB) | Estimated power (W) | Total energy (J) |
|---|---|---|---|---|---|
| $T_{kg}$ | 0.0134 | 1.29 | 0.562 | 2.781 | 0.0373 |
| $T_{puf}$ | 2.066 | 5.75 | 0.96 | 3.062 | 6.3259 |
| $T_h$ | 0.00017 | 2.35 | 0.234 | 2.848 | 0.0005 |
| $T_{kea}$ | 0.054 | 2.046 | 0.191 | 2.829 | 0.1528 |
| $T_{kda}$ | 0.022 | 3.166 | 0.312 | 2.900 | 0.0638 |

**Table 9**
Time taken for each individual operation for secure communication using RPi5 as Drone and i5 PC as GCS.

| Operation | Execution time (s) | Processor utilization (%) | Memory utilization (MB) | Estimated power (W) | Total energy (J) |
|---|---|---|---|---|---|
| $T_{kec}$ | 0.01078 | 1.25 | – | 2.779 | 0.0300 |
| $T_{kdc}$ | 0.006875 | 0.75 | – | 2.747 | 0.0188 |
| $T_{ds}$ | 0.08369 | 4.72 | 0.703 | 2.997 | 0.2508 |
| $T_{dv}$ | 0.02344 | 6.00 | – | 3.078 | 0.0722 |

**Table 10**
Performance analysis of authentication phase of proposed protocol in Raspberry Pi5.

| Author | Total communication cost | Total computations | Estimated computation time (s) | Processor utilization (%) | Estimated power (W) | Total energy (J) |
|---|---|---|---|---|---|---|
| Kumari et al. (2022) | 3172 bytes | $T_{kg} + T_{sg} + T_{verify}$ | 0.117 | 14.32 | 3.5 | 0.409 |
| Mansoor et al. (2024a) | 3732 bytes | $T_{kg} + T_{sg} + T_{verify}$ | 0.085 | 2.93 | 2.885 | 0.245 |
| Ye et al. (2024) | 6318 bytes | $T_{kg} + T_{sg} + T_{verify}$ | 0.076 | 4.84 | 2.995 | 0.2276 |
| Mustafa et al. (2020) | 3600 bytes | $T_{kg} + T_{sg} + T_{verify}$ | 1.495 | 9.7 | 3.31 | 4.948 |
| Choi and Lee (2024) | 3796 bytes | $T_{kg} + T_{sg} + T_{verify}$ | 0.095 | 14.64 | 3.518 | 0.3342 |
| Proposed protocol | 7533 bytes | $T_{kg} + T_h + T_{kea} + T_{kda}$ | 0.08 (w/o serial port monitoring time) | 3.72 | 2.934 | 0.2347 |

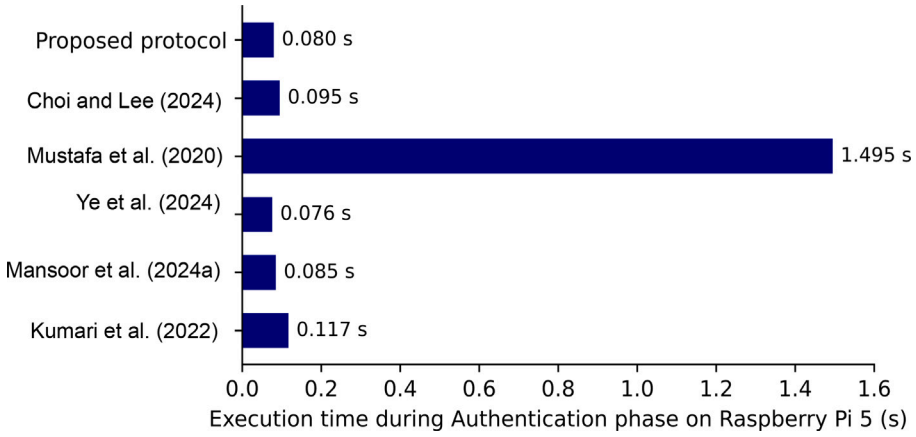



**Fig. 7.** Computation time comparison of the authentication phase with existing methods.

**Table 11**
Performance analysis of communication phase of proposed protocol in Raspberry Pi5.

| Author | Total communication cost | Total computations | Estimated computation time (s) | Processor utilization (%) | Estimated power (W) | Total energy (J) |
|---|---|---|---|---|---|---|
| Kumari et al. (2022) | 180.768 KB | $T_{kg} + T_{enc} + T_{dec}$ | 0.192 | 8.22 | 3.218 | 0.618 |
| Mansoor et al. (2024a) | 1446 bytes | $T_{kg} + T_{enc} + T_{dec}$ | 0.137 | 0.6 | 2.737 | 0.375 |
| Ye et al. (2024) | 3200 bytes | $T_{kg} + T_{encap} + T_{decap}$ | 0.197 | 0.3 | 2.718 | 0.536 |
| Mustafa et al. (2020) | 5136 bytes | $T_{enc} + T_{dec}$ | 0.298 | 15.2 | 3.557 | 1.06 |
| Choi and Lee (2024) | 1664 bytes | $T_{kg} + T_{enc} + T_{dec}$ | 0.125 | 8.2 | 3.216 | 0.402 |
| Proposed protocol | 3188 bytes | $T_{kec} + T_{kdc} + T_{ds} + T_{dv}$ | 0.12 | 12.72 | 3.431 | 0.412 |

drone is very minimal which includes only non-sensitive data like the Kyber public key, timestamp, drone ID, and a challenge value.

### 8.3. Hyperledger fabric performance analysis

To evaluate the system's efficiency, performance testing was carried out using Hyperledger Caliper (Kaushal and Kumar, 2024). The evaluation was conducted in the Ground Control Station (GCS) domain, where the blockchain network consisted of a single Certificate Authority (ca1.gcs.iod.com), one peer node (peer1.gcs.iod.com), and one orderer node (orderer1.gcs.iod.com) configured with the RAFT consensus mechanism. RAFT was chosen for its lightweight, leader-based architecture that eliminates the need for mining, thereby reducing energy consumption and improving efficiency compared to other

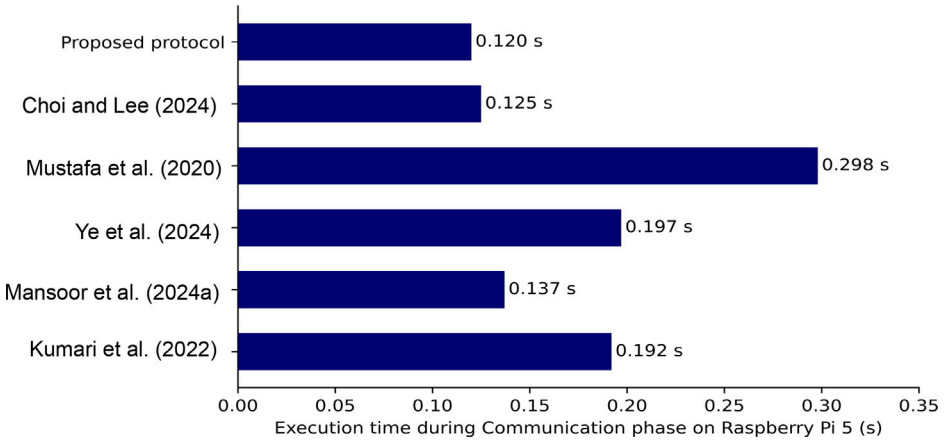
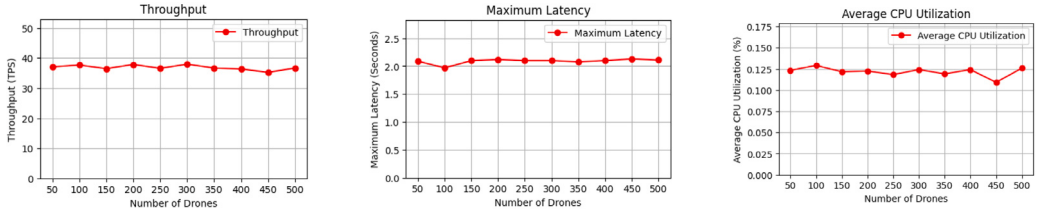
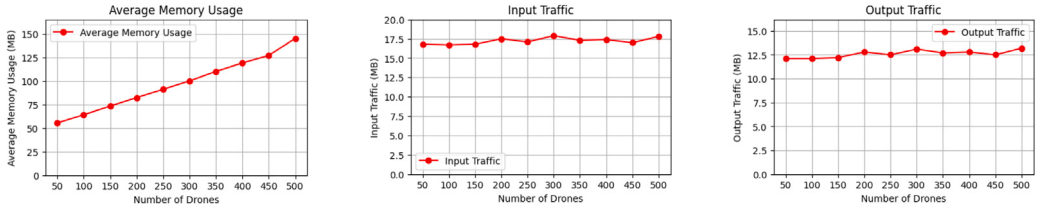



**Fig. 8.** Computation time comparison of the communication phase with existing methods.

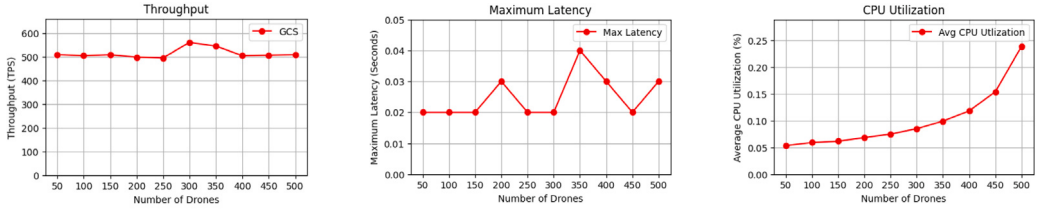



(a) Throughput (b) Maximum latency (c) CPU Utilization

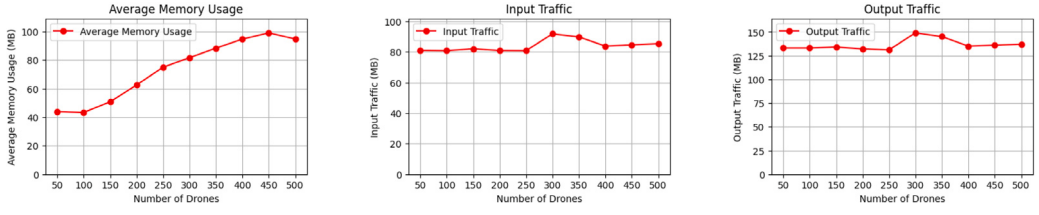



(d) Average memory usage (e) Input traffic (f) Output traffic

**Fig. 9.** Performance evaluation using Caliper for Credential writing (Registration) into the ledger.




(a) Throughput (b) Maximum latency (c) CPU Utilization




(d) Average memory usage (e) Input traffic (f) Output traffic

**Fig. 10.** Performance evaluation using Caliper for Credential reading from the ledger.(a) Throughput.

**Table 12**
Block generation performance evaluation.

| No. of drones | Average blocks generated per minute | Block generation time (s) |
|---|---|---|
| 50 | 245 | 0.245 |
| 100 | 238 | 0.252 |
| 150 | 223 | 0.269 |
| 200 | 213 | 0.282 |
| 250 | 211 | 0.284 |
| 300 | 205 | 0.293 |
| 350 | 168 | 0.357 |
| 400 | 161 | 0.373 |
| 450 | 139 | 0.432 |
| 500 | 127 | 0.472 |

consensus mechanisms. The tests were performed with the number of drones varying from 50 to 500, measuring key parameters such as throughput, latency, CPU utilization, memory usage, and network traffic (input and output) during credential write (Registration) and read operations. As shown in Table 12, the block generation time increased gradually from 0.245 s at 50 drones to 0.472 s at 500 drones, indicating a slight rise in block creation delay as the transaction load increased.

As shown in Fig. 9, during the Credential Write (Registration) phase, the throughput remained stable between 37.1 to 36.7 transactions per second (TPS) even as the number of drones increased from 50 to 500. This indicates that the system maintains consistent performance as the load increases. The maximum latency increased slightly from about 2.09 s to just under 2.11 s, showing that the credential registration delay remains within acceptable limits. CPU usage started at around 0.123 s and slightly increased to approximately 0.126 s, which means the system handled more drones efficiently over time. Memory usage increased linearly from roughly 55.4 MB to 145 MB, which is expected since more credential data is being processed. Input network traffic ranged from 16.8 MB to 17.8 MB, and output traffic ranged from 12.1 MB to 13.2 MB, indicating minimal and stable communication overhead during the registration process.

For the Credential Read phase in Fig. 10, system performance also remained strong and consistent. The throughput stayed around 513 TPS across all drone counts, and the maximum latency was very low ranging from 0.02 to 0.03 s, even at the highest load. CPU usage during read operations started below 0.05 and increased slightly to around 0.23, which remains relatively low. Memory usage increased from approximately 44 MB to 95 MB, demonstrating efficient resource usage as the data volume grew. Input network traffic ranged from 81 MB to 85.4 MB, while output traffic varied between 133 MB and 134 MB, both remaining steady and within manageable limits.

These results confirm that the credential registration latency consistently remained below 2.11 s and the storage overhead increased linearly, but stayed within acceptable limits, reaching a maximum of around 145 MB for 500 drones. The stable throughput, low latency, and efficient utilization of CPU, memory, and network resources during both registration and reading operations demonstrate the system's ability to perform reliably under high-volume conditions. Hence, the proposed solution supports end-to-end credential management with low latency, minimal storage overhead, and consistent performance under heavy workloads, making it suitable for large-scale IoD environments.

## 9. Discussions

### 9.1. Security aspects and alignment with related cryptographic implementations

Our work focuses on secure and lightweight drone communication using quantum-safe algorithms such as Kyber and Dilithium. The study in Sanal et al. (2021) shows that Kyber performs efficiently on low-power ARM-based platforms, similar to the Raspberry Pi devices used in our evaluation. We also use hardware-based key generation to avoid storing static keys, which aligns with the emphasis on physical security and fault tolerance in lightweight ciphers such as LED and HIGHT (Subramanian et al., 2017).

Although our protocol is designed using lattice-based cryptography to ensure resistance against quantum attacks, several studies in related cryptographic domains provide valuable insights into emerging security challenges relevant to resource-constrained environments. Design principles from isogeny-based cryptographic systems, such as scalability and side-channel resistance, are also relevant (Koziel et al., 2018). This is demonstrated by Koziel et al. (2016), which introduces constant-time hardware architectures for Supersingular Isogeny Diffie–Hellman (SIDH) on FPGAs, emphasizing how preventing timing leaks can protect implementations from side-channel attacks, which is a concern also applicable to lattice-based systems. Another work by Jalali et al. (2019) presents optimized SIDH execution on 64-bit ARM processors, demonstrating the need to adapt cryptographic code for modern microarchitectures where physical attacks such as differential power analysis are becoming more effective. A study by Azarderakhsh et al. (2014) explores compact and energy-efficient elliptic curve algorithms designed for highly constrained devices, emphasizing the importance of developing protocols with minimal computational and memory overhead, which aligns with the objectives of our protocol. Also, work by Aghaie et al. (2018) examines error detection schemes in lightweight MixColumns implementations, emphasizing the necessity of fault resilience against injection and tampering attacks in embedded cryptographic hardware.

### 9.2. Fault detection and resilience

Fault detection is crucial for securing embedded cryptographic systems in resource-constrained platforms such as drones. In our proposed quantum-safe drone-to-drone communication protocol, fault resilience is particularly important because both the PUF-based key generation and lattice-based cryptographic operations are sensitive to physical and transient faults. Lightweight fault detection techniques, such as parity-based S-box monitoring, can be adapted to protect control messages and verify internal computations (Kermani and Reyhani-Masoleh, 2006). Similarly, SHA-3 fault detection architectures enhance hash-based integrity verification during message signing and validation (Bayat-Sarmadi et al., 2014). The error detection constructions using CRC and Hamming codes for finite field inversions over GF($2^m$) are applicable to FPGA-based implementations of lattice operations in Kyber and Dilithium (Cintas-Canto et al., 2023a). The fault-resilient designs for code-based post-quantum cryptography demonstrate that fault attacks can compromise signature generation and key encapsulation, emphasizing the need to integrate hardware-level fault detection mechanisms into Dilithium and Kyber computations (Cintas-Canto et al., 2023b). Incorporating secure hardware practices as suggested in the context of medical device security increases the system's trustworthiness and shows the importance of fault resilience in practical deployment scenarios (Kermani et al., 2016). These insights can be incorporated into future versions of our protocol to improve the

**Table 13**
Estimated time and resource usage for authentication using RPi5 with Kyber768.

| Operation | Execution time (s) | Processor utilization (%) | Memory utilization (MB) | Estimated power (W) | Total energy (J) |
|---|---|---|---|---|---|
| $T_{kg}$ | 0.0152 | 1.37 | 0.594 | 2.84 | 0.0432 |
| $T_{puf}$ | 2.128 | 6.02 | 1.02 | 3.11 | 6.6181 |
| $T_h$ | 0.00021 | 2.58 | 0.251 | 2.88 | 0.0006 |
| $T_{kea}$ | 0.0702 | 2.66 | 0.206 | 2.97 | 0.2085 |
| $T_{kda}$ | 0.0286 | 4.12 | 0.334 | 3.05 | 0.0873 |

reliability of cryptographic functions used during authentication and communication between drones in the IoD.

### 9.3. Impact of parameter variants on resource usage and suitability for micro-UAVs

To evaluate the impact of adopting stronger parameter variants in the proposed protocol, we compared performance across both the authentication and secure communication phases on Raspberry Pi 5. As shown in Table 8, the authentication phase using Kyber512 and Dilithium2 showed low resource utilization with encryption taking 0.054 s, decryption 0.022 s, and a maximum memory usage of 0.562 MB during key generation. When moving to Kyber768, Table 13 indicates a slight increase, with encryption taking 0.0702 s and decryption 0.0286 s. The processor usage increased from 2.046 percent to 2.66 percent during encryption and from 3.166 percent to 4.12 percent during decryption. Estimated power consumption increased slightly, reaching 2.97 watts for encryption and 3.05 watts for decryption, while the total energy rose from 0.1528 joules to 0.2085 joules and from 0.0638 joules to 0.0873 joules respectively. Similarly, in the secure communication phase, as seen in Table 9, encryption and decryption using Kyber512 consumed 0.01078 s and 0.006875 s respectively, with no significant memory usage. When Kyber768 was used instead, as shown in Table 14, the encryption and decryption times increased slightly to 0.0140 s and 0.0089 s, along with processor usage increasing from 1.25 percent to 1.63 percent for encryption and from 0.75 percent to 0.98 percent for decryption. In digital signing and verification, shifting from Dilithium2 to Dilithium3 led to execution time increases from 0.08369 s to 0.1172 s for signing and from 0.02344 s to 0.0328 s for verification. The memory usage for signing increased from 0.703 MB to 0.984 MB, and estimated energy for signing increased from 0.2508 joules to 0.3651 joules, demonstrating that stronger cryptographic parameters result in consistent overheads across all metrics.

Despite the slight increase in computational cost and energy usage due to Kyber768 and Dilithium3, the results confirm that the protocol remains feasible for deployment in resource-constrained micro-UAV controllers with less than 1 GB RAM. Across all four tables (Tables 8, 13, 9, 14), memory usage consistently remained below 1.02 MB even under the heaviest operations such as PUF execution and digital signing. Processor utilization remained under 9 percent for all operations, with total energy consumption within an acceptable range for low-power drone hardware. Notably, memory-intensive operations such as PUF-based key generation (2.128 s, 1.02 MB) and digital signing (0.1172 s, 0.984 MB) were still well within the operational limits. Therefore, the results indicate that the proposed protocol is well-suited for micro-UAVs with limited memory and power capacity, making it practical for real-world drone swarm deployments while ensuring strong, quantum-resistant communication without imposing excessive load on the hardware.

### 9.4. Limitations

Despite the strong performance and security guarantees demonstrated by our proposed quantum-secure authentication and communication protocol, several limitations exist. First, the system currently depends on external Arduino hardware for extracting PUF responses, which introduces latency. The serial port reading time for PUF data is 2.26 s on Raspberry Pi 4 and 2.15 s on Raspberry Pi 5, which impacts time-sensitive drone operations. Integrating the PUF module directly into the drone hardware can significantly reduce this latency and improve real-time responsiveness. Second, while the proposed protocol stores credentials on a permissioned blockchain using Kyber encryption to ensure quantum-safe confidentiality, it does not yet fully integrate a quantum-safe blockchain architecture. Adding this feature in the future can help improve security and make the system more reliable. Third, although the protocol has been thoroughly tested in simulated environments using Raspberry Pi boards, it has not yet been evaluated under real-time flight conditions, where network latency, mobility, and environmental factors affect performance. Lastly, the communication cost is slightly higher due to the integration of Kyber and Dilithium, it remains acceptable over 5G networks. Despite these limitations, the protocol successfully combines physical unclonability, post-quantum cryptography, and blockchain-based security into a lightweight solution for next-generation secure drone communications.

## 10. Conclusion and future scope

This paper proposes a quantum-safe drone-to-drone communication protocol for the IoD, focusing on secure authentication and message integrity. The proposed framework integrates PUF for authentication and key generation, lattice-based PQC algorithms such as Kyber for encryption and Dilithium for digital signatures, and a permissioned Hyperledger Fabric blockchain for tamper-resistant credential storage. PUF are utilized to generate unique responses at runtime, which removes the need for static credential storage on drones and mitigates physical extraction risks. The blockchain ensures immutability and decentralized trust during credential management, secured using post-quantum encryption. To validate the protocol's robustness, we conducted a multi-layered security evaluation. At the formal level, BAN logic was applied to verify mutual authentication, key freshness, and belief consistency among drones and the GCS. The protocol goals and message flows were analyzed under formal assumptions, confirming that all entities agree on the shared session keys and authenticity of each other. Also, the protocol was modeled and verified using the Scyther security analysis tool, which simulated adversarial scenarios under the Dolev–Yao model. The results showed that the protocol is secure against a range of classical and quantum-era attacks, including impersonation, replay, message tampering, and MitM attacks. At the informal level, a detailed security mapping was provided against different attacks, verifying confidentiality and unforgeability even under quantum assumptions. The protocol ensures forward secrecy, physical unclonability, and resistance to side-channel threats due to the hardware-based nature of PUF and stateless key generation. The experiment was carried out using Raspberry Pi-based UAVs and an Intel i5-based GCS, and the results were compared with existing authentication and communication methods. The protocol achieves low computation time, while the slight increase in communication cost remains acceptable for drones operating over 5G networks. The experimental results from Hyperledger Fabric confirm that the proposed protocol maintains stable throughput and low latency, even as the number of drones scales up to 500. This indicates

**Table 14**
Estimated time and resource usage for secure communication using RPi5 with Kyber768 and Dilithium3.

| Operation | Execution time (s) | Processor utilization (%) | Memory utilization (MB) | Estimated power (W) | Total energy (J) |
|---|---|---|---|---|---|
| $T_{kec}$ | 0.0140 | 1.63 | 0.312 | 2.918 | 0.0397 |
| $T_{kdc}$ | 0.0089 | 0.98 | 0.274 | 2.884 | 0.0275 |
| $T_{ds}$ | 0.1172 | 6.61 | 0.984 | 3.147 | 0.3651 |
| $T_{dv}$ | 0.0328 | 8.40 | 0.612 | 3.232 | 0.1058 |

strong scalability, making the protocol suitable for large-scale IoD deployments. Also, the decentralized blockchain structure avoids single points of failure, ensuring reliability and resilience in dynamic drone environments. Compared to existing PQC-based solutions that often depend on static key storage and lack secure credential management, our protocol ensures dynamic key generation, physical unclonability, and secure storage using blockchain.

This work can be extended in the future by:

- Extending the protocol for secure multi-hop communication among drone swarms.
- Testing the system in a real-time, 5G-enabled IoD environment to validate practical performance.
- Integrating with decentralized quantum-safe blockchain systems for broader interoperability.

## CRediT authorship contribution statement

**Jithu Vijay V.P.:** Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Shahanas I.N.:** Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Sabu M. Thampi:** Writing – review & editing, Supervision, Software, Project administration, Methodology, Investigation, Funding acquisition, Conceptualization. **Aiswarya S. Nair:** Software, Resources, Methodology.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment

This research was supported by Quantumweave Intelligence Private Limited, Trivandrum, Kerala, India. The authors gratefully acknowledge the organization's support and resources that contributed to the successful completion of this work.

## Appendix. SPDL model for our proposed a quantum-secure digital signature-based communication protocol for the Internet of Drones (IoD)

```
protocol DroneSystem(GCS, Drone1, Drone2) {

  // Ground Control Station (GCS) role
  role GCS {
    fresh kyberKeyGCS: Nonce; // Public key for
        encryption
    fresh kyberKeyDrone1: Nonce;
    fresh kyberKeyDrone2: Nonce;
    fresh challengeDrone1: Nonce;
    fresh challengeDrone2: Nonce;
    fresh ackSecure: Nonce; // Acknowledgment for secure
        communication
    var responseDrone1: Nonce;
    var responseDrone2: Nonce;
    var droneID1: Nonce;
    var droneID2: Nonce;
    fresh timestamp1: Nonce;
    fresh timestamp2: Nonce;
    fresh tempKey: Nonce;

    // Registration for Drone1
    recv_1(Drone1, GCS, droneID1, kyberKeyDrone1);
    send_2(GCS, Drone1, tempKey, {challengeDrone1}
        kyberKeyDrone1);
    recv_3(Drone1, GCS, {responseDrone1}tempKey);
    claim(GCS, Commit, Drone1, (droneID1, kyberKeyDrone1,
        challengeDrone1, responseDrone1));
    send_4(GCS, Drone1, kyberKeyGCS);

    // Registration for Drone2
    recv_5(Drone2, GCS, droneID2, kyberKeyDrone2);
    send_6(GCS, Drone2, tempKey, {challengeDrone2}
        kyberKeyDrone2);
    recv_7(Drone2, GCS, {responseDrone2}tempKey);
    claim(GCS, Commit, Drone2, (droneID2, kyberKeyDrone2,
        challengeDrone2, responseDrone2));
    send_8(GCS, Drone2, kyberKeyGCS);

    // GCS receives forwarded request from Drone2
    recv_10(Drone2, GCS, droneID1, droneID2, timestamp1,
        timestamp2);
    claim(GCS, Commit, Drone2, (droneID1, droneID2,
        timestamp1, timestamp2));

    // GCS verifies Drone2's challenge--response pair
    send_11(GCS, Drone2, {challengeDrone2}kyberKeyDrone2)
        ;
    recv_12(Drone2, GCS, {responseDrone2}kyberKeyGCS);
    claim(GCS, Commit, Drone2, (challengeDrone2,
        responseDrone2));

    // GCS provides details of Drone1 to Drone2
    send_13(GCS, Drone2, {kyberKeyDrone1, challengeDrone1
        , responseDrone1, droneID1}kyberKeyDrone2);
    claim(GCS, Reachable);
  }

  // Drone1 role
  role Drone1 {
    fresh kyberKeyDrone1: Nonce; // Public key for
        encryption
    fresh dilithiumPvtKeyDrone1: Nonce; // Private
        signing key
    var kyberKeyDrone2: Nonce; // Public key of Drone2
    fresh droneID1: Nonce; // ID assigned by GCS
    fresh timestamp1: Nonce; // Timestamp for freshness
    fresh secureMessage: Nonce; // Message for secure
        communication
    fresh challengeDrone1: Nonce; // Challenge received
        from GCS
    fresh responseDrone1: Nonce; // Response to the
        challenge
    fresh ackSecure: Nonce; // Acknowledgment for secure
        communication
    var kyberKeyGCS: Nonce;
    var tempKey: Nonce;

    // Registration
    send_1(Drone1, GCS, droneID1, kyberKeyDrone1);
    recv_2(GCS, Drone1, tempKey, {challengeDrone1}
        kyberKeyDrone1);
    send_3(Drone1, GCS, {responseDrone1}tempKey);
    recv_4(GCS, Drone1, kyberKeyGCS);
    claim(Drone1, Commit, GCS, (kyberKeyGCS, droneID1,
        challengeDrone1, responseDrone1));

    // Drone1 initiates communication with Drone2
    send_9(Drone1, Drone2, droneID1, timestamp1);
```

```
    // Drone1 receives a challenge from Drone2
    recv_14(Drone2, Drone1, {challengeDrone1,
        kyberKeyDrone2}kyberKeyDrone1);

    // Drone1 sends response and public key to Drone2
    send_15(Drone1, Drone2, {responseDrone1,
        dilithiumPvtKeyDrone1}kyberKeyDrone2);

    // Drone1 receives acknowledgment from Drone2
    recv_16(Drone2, Drone1, {ackSecure, kyberKeyDrone2}
        kyberKeyDrone1);
    claim(Drone1, Commit, Drone2, (ackSecure,
        kyberKeyDrone2, dilithiumPvtKeyDrone1));

    // Sending secure signed and encrypted message
    send_17(Drone1, Drone2, {{secureMessage}
        dilithiumPvtKeyDrone1, secureMessage}
        kyberKeyDrone2);
    claim(Drone1, Commit, Drone2, (secureMessage,
        dilithiumPvtKeyDrone1, kyberKeyDrone2));

    claim(Drone1, Reachable);
    claim(Drone1, Secret, secureMessage);
}

// Drone2 role
role Drone2 {
    fresh kyberKeyDrone2: Nonce; // Public key for
        encryption
    fresh droneID2: Nonce; // ID assigned by GCS
    fresh timestamp2: Nonce; // Timestamp for freshness
    var droneID1: Nonce; // ID of Drone1
    var timestamp1: Nonce; // Timestamp received from
        Drone1
    fresh challengeDrone2: Nonce; // Challenge received
        from GCS
    fresh responseDrone2: Nonce; // Response to the
        challenge
    var responseDrone1: Nonce; // Response received from
        Drone1
    var challengeDrone1: Nonce; // Challenge sent by GCS
        for Drone1
    fresh ackSecure: Nonce; // Acknowledgment for secure
        communication
    var secureMessage: Nonce; // Message received from
        Drone1
    var dilithiumPvtKeyDrone1: Nonce; // Private signing
        key of Drone1
    var kyberKeyDrone1: Nonce; // Public key of Drone1
    var kyberKeyGCS: Nonce; // Public key of GCS
    var tempKey: Nonce; // Temporary key used for
        challenge--response encryption

    // Registration
    send_5(Drone2, GCS, droneID2, kyberKeyDrone2);
    recv_6(GCS, Drone2, tempKey, {challengeDrone2}
        kyberKeyDrone2);
    send_7(Drone2, GCS, {responseDrone2}tempKey);
    recv_8(GCS, Drone2, kyberKeyGCS);
    claim(Drone2, Commit, GCS, (kyberKeyGCS, droneID2,
        challengeDrone2, responseDrone2));

    // Receiving request from Drone1
    recv_9(Drone1, Drone2, droneID1, timestamp1);
    send_10(Drone2, GCS, droneID1, droneID2, timestamp1,
        timestamp2);

    // Receiving verification from GCS
    recv_11(GCS, Drone2, {challengeDrone2}kyberKeyDrone2)
        ;
    send_12(Drone2, GCS, {responseDrone2}kyberKeyGCS);

    // Receiving Drone1 details from GCS
    recv_13(GCS, Drone2, {kyberKeyDrone1, challengeDrone1
        , responseDrone1, droneID1}kyberKeyDrone2);

    // Sending challenge to Drone1
    send_14(Drone2, Drone1, {challengeDrone1,
        kyberKeyDrone2}kyberKeyDrone1);

    // Receiving response from Drone1
    recv_15(Drone1, Drone2, {responseDrone1,
        dilithiumPvtKeyDrone1}kyberKeyDrone2);
    claim(Drone2, Commit, Drone1, (challengeDrone1,
        responseDrone1, dilithiumPvtKeyDrone1));

    // Sending acknowledgment to Drone1
    send_16(Drone2, Drone1, {ackSecure, kyberKeyDrone2}
        kyberKeyDrone1);
    claim(Drone2, Commit, Drone1, (ackSecure,
        kyberKeyDrone2, dilithiumPvtKeyDrone1));

    // Receiving secure message
    recv_17(Drone1, Drone2, {{secureMessage}
        dilithiumPvtKeyDrone1, secureMessage}
        kyberKeyDrone2);
    claim(Drone2, Commit, Drone1, (secureMessage,
        dilithiumPvtKeyDrone1, kyberKeyDrone2));
    claim(Drone2, Secret, secureMessage);
    claim(Drone2, Niagree);
  }
}
```

## Data availability

The authors are unable or have chosen not to specify which data has been used.

## References


Abbood, A.N., Al-Shammri, F., Alzamili, Z., Al-Shareeda, M., Almaiah, M., Alali, R., 2025. Investigating quantum-resilient security mechanisms for flying ad-hoc networks (FANETs). J. Robot. Control. (JRC) 6, http://dx.doi.org/10.18196/jrc.v6i1.25351.

Aghaie, A., Kermani, M.M., Azarderakhsh, R., 2018. Comparative realization of error detection schemes for implementations of mixcolumns in lightweight cryptography. In: Proceedings of the 15th ACM International Conference on Computing Frontiers. CF '18, Association for Computing Machinery, New York, NY, USA, pp. 200–203. http://dx.doi.org/10.1145/3203217.3203258.

Ali, G., Samuel, A., Kabiito, S.P., Morish, Z., Thomas, A., Robert, W., Denis, A., Sallam, M., Mijwil, M.M., Ayad, J., et al., 2025. Integration of artificial intelligence, blockchain, and quantum cryptography for securing the industrial internet of things (iIoT): Recent advancements and future trends. Appl. Data Sci. Anal. 2025, 19–82. http://dx.doi.org/10.58496/ADSA/2025/004, URL https://mesopotamian.press/journals/index.php/ADSA/article/view/757.

Alladi, T., Naren, Bansal, G., Chamola, V., Guizani, M., 2020. SecAuthUAV: A novel authentication scheme for UAV-ground station and UAV-UAV communication. IEEE Trans. Veh. Technol. 69 (12), 15068–15077. http://dx.doi.org/10.1109/TVT.2020.3033060.

Alzahrani, B., Oubbati, O.S., Barnawi, A., Atiquzzaman, M., Alghazzawi, D., 2020. UAV assistance paradigm: State-of-the-art in applications and challenges. J. Netw. Comput. Appl. 166, 102706. http://dx.doi.org/10.1016/j.jnca.2020.102706, URL https://www.sciencedirect.com/science/article/pii/S1084804520301806.

Anon, 2025. Advanced 6G network protection using quantum key distribution: A systematic review. Babylon. J. Netw. 2025, 80–96. http://dx.doi.org/10.58496/BJN/2025/007, URL https://mesopotamian.press/journals/index.php/BJN/article/view/854.

Azarderakhsh, R., Järvinen, K.U., Mozaffari-Kermani, M., 2014. Efficient algorithm and architecture for elliptic curve cryptography for extremely constrained secure applications. IEEE Trans. Circuits Syst. I. Regul. Pap. 61 (4), 1144–1155. http://dx.doi.org/10.1109/TCSI.2013.2283691.

Bayat-Sarmadi, S., Mozaffari-Kermani, M., Reyhani-Masoleh, A., 2014. Efficient and concurrent reliable realization of the secure cryptographic SHA-3 algorithm. IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 33 (7), 1105–1109. http://dx.doi.org/10.1109/TCAD.2014.2307002.

Blazy, O., Bonnefoi, P.-F., Conchon, E., Sauveron, D., Akram, R.N., Markantonakis, K., Mayes, K., Chaumette, S., 2017. An efficient protocol for UAS security. In: 2017 Integrated Communications, Navigation and Surveillance Conference. ICNS, pp. 1–21. http://dx.doi.org/10.1109/ICNSURV.2017.8011987.

Boccadoro, P., Striccoli, D., Grieco, L.A., 2021. An extensive survey on the internet of drones. Ad Hoc Networks 122, 102600. http://dx.doi.org/10.1016/j.adhoc.2021.102600, URL https://www.sciencedirect.com/science/article/pii/S1570870521001335.

Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehle, D., 2018. CRYSTALS - kyber: A CCA-secure module-lattice-based KEM. In: 2018 IEEE European Symposium on Security and Privacy. EuroS&P, pp. 353–367. http://dx.doi.org/10.1109/EuroSP.2018.00032.

Burhanuddin, M., 2023. Secure and scalable quantum cryptographic algorithms for next-generation computer networks. Khwarizmia 2023, 95–102. http://dx.doi.org/10.70470/KHWARIZMIA/2023/009, URL https://peninsula-press.ae/Journals/index.php/KHWARIZMIA/article/view/22.

Chen, Y.-J., Hsu, C.-L., Lin, T.-W., Lee, J.-S., 2024. Design and evaluation of device authentication and secure communication system with PQC for AIoT environments. Electronics 13 (8), URL https://www.mdpi.com/2079-9292/13/8/1575.

Choi, J., Lee, J., 2024. Secure and scalable internet of things model using post-quantum macsec. Appl. Sci. 14 (10), http://dx.doi.org/10.3390/app14104215, URL https://www.mdpi.com/2076-3417/14/10/4215.

Cintas-Canto, A., Kermani, M.M., Azarderakhsh, R., 2023a. Error detection constructions for ITA finite field inversions over GF($2^m$) on FPGA using CRC and hamming codes. IEEE Trans. Reliab. 72 (2), 651–661. http://dx.doi.org/10.1109/TR.2022.3216014.

Cintas-Canto, A., Mozaffari-Kermani, M., Azarderakhsh, R., 2023b. Reliable code-based post-quantum cryptographic algorithms through fault detection on FPGA. In: 2023 IEEE Nordic Circuits and Systems Conference. NorCAS, pp. 1–5. http://dx.doi.org/10.1109/NorCAS58970.2023.10305475.

Derhab, A., Cheikhrouhou, O., Allouch, A., Koubaa, A., Qureshi, B., Ferrag, M.A., Maglaras, L., Khan, F.A., 2023. Internet of drones security: Taxonomies, open issues, and future directions. Veh. Commun. 39, 100552. http://dx.doi.org/10.1016/j.vehcom.2022.100552, URL https://www.sciencedirect.com/science/article/pii/S2214209622000997.

Ding, J., Alsayigh, S., Lancrenon, J., RV, S., Snook, M., 2017. Provably secure password authenticated key exchange based on RLWE for the post-quantum world. In: Handschuh, H. (Ed.), Topics in Cryptology – CT-RSA 2017. Springer International Publishing, Cham, pp. 183–204.

Duraibi, S., Alashjaee, A.M., 2025. Lightweight post-quantum secure communication protocol for IoT devices using code-based cryptography. IEEE Trans. Consum. Electron. http://dx.doi.org/10.1109/TCE.2025.3611586, 1–1.

El-Zawawy, M.A., Brighente, A., Conti, M., 2022. SETCAP: Service-based energy-efficient temporal credential authentication protocol for internet of drones. Comput. Netw. 206, 108804. http://dx.doi.org/10.1016/j.comnet.2022.108804, URL https://www.sciencedirect.com/science/article/pii/S1389128622000305.

Hadi, H.J., Cao, Y., Nisa, K.U., Jamil, A.M., Ni, Q., 2023. A comprehensive survey on security, privacy issues and emerging defence technologies for UAVs. J. Netw. Comput. Appl. 213, 103607. http://dx.doi.org/10.1016/j.jnca.2023.103607, URL https://www.sciencedirect.com/science/article/pii/S1084804523000267.

Hussain, S., Chaudhry, S.A., Alomari, O.A., Alsharif, M.H., Khan, M.K., Kumar, N., 2021. Amassing the security: An ECC-based authentication scheme for internet of drones. IEEE Syst. J. 15 (3), 4431–4438. http://dx.doi.org/10.1109/JSYST.2021.3057047.

IBM, 2022. Lattice-based cryptography. URL https://research.ibm.com/projects/lattice-based-cryptography. (Accessed 07 May 2025).

Islam, N., Hossain, M.K., Ali, G.G.M.N., Chong, P.H.J., 2016. An expedite group key establishment protocol for flying ad-hoc network(FANET). In: 2016 5th International Conference on Informatics, Electronics and Vision. ICIEV, pp. 312–315. http://dx.doi.org/10.1109/ICIEV.2016.7760017.

Jackson, K.A., Miller, C.A., Wang, D., 2024. Evaluating the security of RYSTALS-dilithium in the quantum random oracle model. In: Advances in Cryptology – EUROCRYPT 2024: 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26–30, 2024, Proceedings, Part VI. Springer-Verlag, Berlin, Heidelberg, pp. 418–446. http://dx.doi.org/10.1007/978-3-031-58751-1_15.

Jalali, A., Azarderakhsh, R., Kermani, M.M., Jao, D., 2019. Supersingular isogeny diffie-hellman key exchange on 64-bit ARM. IEEE Trans. Dependable Secur. Comput. 16 (5), 902–912. http://dx.doi.org/10.1109/TDSC.2017.2723891.

Jithu Vijay, V.P., Thampi, S.M., 2025. Navigating the skies: Exploring dynamic access control in the internet of drones (IoD). In: Thampi, S.M., Thomas, T., Mukherjee, P. (Eds.), Securing the Connected World: Exploring Emerging Threats and Innovative Solutions. Springer Nature Switzerland, Cham, pp. 345–380. http://dx.doi.org/10.1007/978-3-031-82826-3_10.

Kaushal, R.K., Kumar, N., 2024. Exploring hyperledger caliper benchmarking tool to measure the performance of blockchain based solutions. In: 2024 11th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions). ICRITO, pp. 1–6. http://dx.doi.org/10.1109/ICRITO61523.2024.10522188.

Kermani, M.M., Azarderakhsh, R., Mirakhorli, M., 2016. Education and research integration of emerging multidisciplinary medical devices security. Rochester Institute of Technology Repository. Accessed from https://repository.rit.edu/other/860.

Kermani, M.M., Reyhani-Masoleh, A., 2006. Parity-based fault detection architecture of S-box for advanced encryption standard. In: 2006 21st IEEE International Symposium on Defect and Fault Tolerance in VLSI Systems. pp. 572–580. http://dx.doi.org/10.1109/DFT.2006.50.

Khan, M.A., Javaid, S., Mohsan, S.A.H., Tanveer, M., Ullah, I., 2024. Future-proofing security for UAVs with post-quantum cryptography: A review. IEEE Open J. Commun. Soc. 5, 6849–6871. http://dx.doi.org/10.1109/OJCOMS.2024.3486649.

Ko, Y., Kim, J., Duguma, D.G., Astillo, P.V., You, I., Pau, G., 2021. Drone secure communication protocol for future sensitive applications in military zone. Sensors 21 (6), http://dx.doi.org/10.3390/s21062057, URL https://www.mdpi.com/1424-8220/21/6/2057.

Koziel, B., Azarderakhsh, R., Kermani, M.M., 2018. A high-performance and scalable hardware architecture for isogeny-based cryptography. IEEE Trans. Comput. 67 (11), 1594–1609. http://dx.doi.org/10.1109/TC.2018.2815605.

Koziel, B., Azarderakhsh, R., Mozaffari-Kermani, M., 2016. Fast hardware architectures for supersingular isogeny diffie-hellman key exchange on FPGA. In: Dunkelman, O., Sanadhya, S.K. (Eds.), Progress in Cryptology – INDOCRYPT 2016. Springer International Publishing, Cham, pp. 191–206. http://dx.doi.org/10.1007/978-3-319-49890-4_11.

Kriz, V., Gabrlik, P., 2015. UranusLink - communication protocol for UAV with small overhead and encryption ability. IFAC-PapersOnLine 48 (4), 474–479. http://dx.doi.org/10.1016/j.ifacol.2015.07.080, 13th IFAC and IEEE Conference on Programmable Devices and Embedded Systems. URL https://www.sciencedirect.com/science/article/pii/S2405896315008551.

Kumar, M., Mondal, B., 2024. Study on implementation of shor's factorization algorithm on quantum computer. SN Comput. Sci. 5 (4), 413. http://dx.doi.org/10.1007/s42979-024-02771-y.

Kumari, S., Singh, M., Singh, R., Tewari, H., 2022. A post-quantum lattice based lightweight authentication and code-based hybrid encryption scheme for IoT devices. Comput. Netw. 217, 109327. http://dx.doi.org/10.1016/j.comnet.2022.109327, URL https://www.sciencedirect.com/science/article/pii/S138912862200367X.

Mansoor, K., Afzal, M., Iqbal, W., Abbas, Y., 2024b. Securing the future: exploring post-quantum cryptography for authentication and user privacy in IoT devices. Clust. Comput. 28 (2), 93. http://dx.doi.org/10.1007/s10586-024-04799-4.

Mansoor, K., Afzal, M., Iqbal, W., Abbas, Y., Mussiraliyeva, S., Chehri, A., 2024a. PQCAIE: Post quantum cryptographic authentication scheme for IoT-based e-health systems. Internet Things 27, 101228. http://dx.doi.org/10.1016/j.iot.2024.101228, URL https://www.sciencedirect.com/science/article/pii/S2542660524001690.

Maxa, J.-A., Ben Mahmoud, M.S., Larrieu, N., 2016. Extended verification of secure UAANET routing protocol. In: 2016 IEEE/AIAA 35th Digital Avionics Systems Conference. DASC, pp. 1–16. http://dx.doi.org/10.1109/DASC.2016.7777970.

Mishra, D., Singh, M., Rewal, P., Pursharthi, K., Kumar, N., Barnawi, A., Rathore, R.S., 2023. Quantum-safe secure and authorized communication protocol for internet of drones. IEEE Trans. Veh. Technol. 72 (12), 16499–16507. http://dx.doi.org/10.1109/TVT.2023.3292169.

Mustafa, I., Khan, I.U., Aslam, S., Sajid, A., Mohsin, S.M., Awais, M., Qureshi, M.B., 2020. A lightweight post-quantum lattice-based RSA for secure communications. IEEE Access 8, 99273–99285. http://dx.doi.org/10.1109/ACCESS.2020.2995801.

Nair, A.S., Thampi, S.M., P., J.V.V., 2025. SoCoMNNet: A SocioCognitive and memristive neural network-based context-aware GPS spoofing detection and mitigation in the internet of drones. Veh. Commun. 56, 100980. http://dx.doi.org/10.1016/j.vehcom.2025.100980, URL https://www.sciencedirect.com/science/article/pii/S221420962500107X.

Nguyen, T.-H., Dam, D.-T., Duong, P.-P., Kieu-Do-Nguyen, B., Pham, C.-K., Hoang, T.-T., 2025. Efficient hardware implementation of the lightweight CRYSTALS-kyber. IEEE Trans. Circuits Syst. I. Regul. Pap. 72 (2), 610–622. http://dx.doi.org/10.1109/TCSI.2024.3443238.

Nikooghadam, M., Amintoosi, H., Islam, S.H., Moghadam, M.F., 2021. A provably secure and lightweight authentication scheme for internet of drones for smart city surveillance. J. Syst. Archit. 115, 101955. http://dx.doi.org/10.1016/j.sysarc.2020.101955, URL https://www.sciencedirect.com/science/article/pii/S138376212030206X.

Ojetunde, B., Kurihara, T., Yano, K., Sakano, T., Yokoyama, H., 2025. A practical implementation of post-quantum cryptography for secure wireless communication. Preprints http://dx.doi.org/10.20944/preprints202505.0990.v1.

Parameswarath, R.P., Wang, C., Sikdar, B., 2025. Quantum-safe authentication protocol for IoT-enabled transportation systems. IEEE Trans. Ind. Appl. 1–14. http://dx.doi.org/10.1109/TIA.2025.3585861.

Prajapat, S., Rana, A., Kumar, P., Das, A.K., 2024. Quantum safe lightweight encryption scheme for secure data sharing in internet of nano things. Comput. Electr. Eng. 117, 109253. http://dx.doi.org/10.1016/j.compeleceng.2024.109253, URL https://www.sciencedirect.com/science/article/pii/S0045790624001812.

Pratama, D., Moon, J., Ari Laksmono, A.M., Yun, D., Iqbal, M., Jeong, B., Ji, J.H., Kim, H., 2024. Behind the wings: The case of reverse engineering and drone hijacking in DJI enhanced wi-fi protocol. In: 2024 International Conference on Platform Technology and Service. PlatCon, pp. 127–132. http://dx.doi.org/10.1109/PlatCon63925.2024.10830741.

Pu, C., Wall, A., Choo, K.-K.R., Ahmed, I., Lim, S., 2022. A lightweight and privacy-preserving mutual authentication and key agreement protocol for internet of drones environment. IEEE Internet Things J. 9 (12), 9918–9933. http://dx.doi.org/10.1109/JIOT.2022.3163367.

Sanal, P., Karagoz, E., Seo, H., Azarderakhsh, R., Mozaffari-Kermani, M., 2021. Kyber on ARM64: Compact implementations of kyber on 64-bit ARM cortex-a processors. In: Garcia-Alfaro, J., Li, S., Poovendran, R., Debar, H., Yung, M. (Eds.), Security and Privacy in Communication Networks. Springer International Publishing, Cham, pp. 424–440. http://dx.doi.org/10.1007/978-3-030-90022-9_23.

Semal, B., Markantonakis, K., Akram, R.N., 2018. A certificateless group authenticated key agreement protocol for secure communication in untrusted UAV networks. In: 2018 IEEE/AIAA 37th Digital Avionics Systems Conference. DASC, pp. 1–8. http://dx.doi.org/10.1109/DASC.2018.8569730.

Seo, S.-H., Won, J., Bertino, E., Kang, Y., Choi, D., 2016. A security framework for a drone delivery service. In: Proceedings of the 2nd Workshop on Micro Aerial Vehicle Networks, Systems, and Applications for Civilian Use. DroNet '16, Association for Computing Machinery, New York, NY, USA, pp. 29–34. http://dx.doi.org/10.1145/2935620.2935629.

Shahzadi, R., Ali, M., Khan, H.Z., Naeem, M., 2021. UAV assisted 5G and beyond wireless networks: A survey. J. Netw. Comput. Appl. 189, 103114. http://dx.doi.org/10.1016/j.jnca.2021.103114, URL https://www.sciencedirect.com/science/article/pii/S108480452100134X.

Sharma, A., Rani, S., 2025. Post-quantum cryptography (PQC) for IoT-consumer electronics devices integrated with deep learning. IEEE Trans. Consum. Electron. 71 (2), 4925–4933. http://dx.doi.org/10.1109/TCE.2025.3569904.

Shen, S., Yang, H., Dai, W., Zhang, H., Liu, Z., Zhao, Y., 2024. High-throughput GPU implementation of dilithium post-quantum digital signature. IEEE Trans. Parallel Distrib. Syst. 35 (11), 1964–1976. http://dx.doi.org/10.1109/TPDS.2024.3453289.

Singamaneni, K.K., Muhammad, G., 2024. A novel integrated quantum-resistant cryptography for secure scientific data exchange in ad hoc networks. Ad Hoc Networks 164, 103607. http://dx.doi.org/10.1016/j.adhoc.2024.103607, URL https://www.sciencedirect.com/science/article/pii/S157087052400218X.

Srinivas, J., Das, A.K., Kumar, N., Rodrigues, J.J.P.C., 2019. TCALAS: Temporal credential-based anonymous lightweight authentication scheme for internet of drones environment. IEEE Trans. Veh. Technol. 68 (7), 6903–6916. http://dx.doi.org/10.1109/TVT.2019.2911672.

Subramani, S., M., S., A., K., Svn, S.K., 2025. Review of security methods based on classical cryptography and quantum cryptography. Cybern. Syst. 56 (3), 302–320. http://dx.doi.org/10.1080/01969722.2023.2166261, arXiv:https://doi.org/10.1080/01969722.2023.2166261.

Subramanian, S., Mozaffari-Kermani, M., Azarderakhsh, R., Nojoumian, M., 2017. Reliable hardware architectures for cryptographic block ciphers LED and HIGHT. IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 36 (10), 1750–1758. http://dx.doi.org/10.1109/TCAD.2017.2661811.

Tambe-Jagtap, S.N., 2023. A survey of cryptographic algorithms in cybersecurity: From classical methods to quantum-resistant solutions. Shifra 2023, 43–52. http://dx.doi.org/10.70470/SHIFRA/2023/006, URL https://peninsula-press.ae/Journals/index.php/SHIFRA/article/view/30.

Tanveer, M., Aldosary, A., Khokhar, S.-u.-d., Das, A.K., Aldossari, S.A., Chaudhry, S.A., 2024. PAF-IoD: PUF-enabled authentication framework for the internet of drones. IEEE Trans. Veh. Technol. 73 (7), 9560–9574. http://dx.doi.org/10.1109/TVT.2024.3365992.

Tanveer, M., Alkhayyat, A., Naushad, A., Khan, A.U., Kumar, N., Alharbi, A.G., 2022. RUAM-IoD: A robust user authentication mechanism for the internet of drones. IEEE Access 10, 19836–19851. http://dx.doi.org/10.1109/ACCESS.2022.3149376.

Tawfeeq, N.H., Yousif, M., Al-Shareeda, M.A., Almaiah, M.A., Shehab, R., 2025. Lightweight and quantum-resistant authentication for the internet of drones (IoD) using dilithium signatures.. Int. J. Innov. Res. Sci. Stud. 8 (2), 2842–2853. http://dx.doi.org/10.53894/ijirss.v8i2.5825, URL https://www.ijirss.com/index.php/ijirss/article/view/5825.

Tian, Y., Yuan, J., Song, H., 2019. Efficient privacy-preserving authentication framework for edge-assisted internet of drones. J. Inf. Secur. Appl. 48, 102354. http://dx.doi.org/10.1016/j.jisa.2019.06.010, URL https://www.sciencedirect.com/science/article/pii/S2214212618307038.

Vinoth, P., Kumar, D., Guhan, M., Archana, M., Hari, S., 2024. A secure authentication mechanism for IOT devices using hyperledger fabric. ISBN: 978-981-97-1840-5, pp. 357–372. http://dx.doi.org/10.1007/978-981-97-1841-2_27.

Wang, G., Lim, K., Lee, B.-S., Ahn, J.Y., 2017. Handover key management in an LTE-based unmanned aerial vehicle control network. In: 2017 5th International Conference on Future Internet of Things and Cloud Workshops. FiCloudW, pp. 200–205. http://dx.doi.org/10.1109/FiCloudW.2017.81.

Wang, X., Zhao, Z., Yi, L., Ning, Z., Guo, L., Yu, F.R., Guo, S., 2024. A survey on security of UAV swarm networks: Attacks and countermeasures. ACM Comput. Surv. 57 (3), http://dx.doi.org/10.1145/3703625.

Wazid, M., Das, A.K., Kumar, N., Vasilakos, A.V., Rodrigues, J.J.P.C., 2019. Design and analysis of secure lightweight remote user authentication and key agreement scheme in internet of drones deployment. IEEE Internet Things J. 6 (2), 3572–3584. http://dx.doi.org/10.1109/JIOT.2018.2888821.

Wazid, M., Das, A.K., Lee, J.-H., 2018. Authentication protocols for the internet of drones: taxonomy, analysis and future directions. J. Ambient. Intell. Humaniz. Comput. http://dx.doi.org/10.1007/s12652-018-1006-x.

Wei, X., Ma, J., Sun, C., 2024. A survey on security of unmanned aerial vehicle systems: Attacks and countermeasures. IEEE Internet Things J. 11 (21), 34826–34847. http://dx.doi.org/10.1109/JIOT.2024.3429111.

Won, J., Seo, S.-H., Bertino, E., 2015. A secure communication protocol for drones and smart objects. In: Proceedings of the 10th ACM Symposium on Information, Computer and Communications Security. In: ASIA CCS '15, Association for Computing Machinery, New York, NY, USA, pp. 249–260. http://dx.doi.org/10.1145/2714576.2714616.

Yaacoub, J.-P., Noura, H., Salman, O., Chehab, A., 2020. Security analysis of drones systems: Attacks, limitations, and recommendations. Internet Things 11, 100218. http://dx.doi.org/10.1016/j.iot.2020.100218, URL https://www.sciencedirect.com/science/article/pii/S2542660519302112.

Yahuza, M., Idris, M.Y.I., Wahab, A.W.A., Nandy, T., Ahmedy, I.B., Ramli, R., 2021. An edge assisted secure lightweight authentication technique for safe communication on the internet of drones network. IEEE Access 9, 31420–31440. http://dx.doi.org/10.1109/ACCESS.2021.3060420.

Ye, Z., Song, R., Zhang, H., Chen, D., Cheung, R.C.-C., Huang, K., 2024. A highly-efficient lattice-based post-quantum cryptography processor for IoT applications. IACR Trans. Cryptogr. Hardw. Embed. Syst. 2024 (2), 130–153. http://dx.doi.org/10.46586/tches.v2024.i2.130-153, URL https://tches.iacr.org/index.php/TCHES/article/view/11423.

Yu, S., Das, A.K., Park, Y., Lorenz, P., 2022. SLAP-IoD: Secure and lightweight authentication protocol using physical unclonable functions for internet of drones in smart city environments. IEEE Trans. Veh. Technol. 71 (10), 10374–10388. http://dx.doi.org/10.1109/TVT.2022.3188769.

Zhang, Y., He, D., Li, L., Chen, B., 2020. A lightweight authentication and key agreement scheme for internet of drones. Comput. Commun. 154, 455–464. http://dx.doi.org/10.1016/j.comcom.2020.02.067, URL https://www.sciencedirect.com/science/article/pii/S0140366419319358.

Zhang, N., Jiang, Q., Li, L., Ma, X., Ma, J., 2021. An efficient three-factor remote user authentication protocol based on BPV-FourQ for internet of drones. Peer-To-Peer Netw. Appl. 14 (5), 3319–3332. http://dx.doi.org/10.1007/s12083-021-01130-5.

Zhao, T., Zhang, Y., Wang, M., Feng, W., Cao, S., Wang, G., 2025. A critical review on the battery system reliability of drone systems. Drones 9 (8), http://dx.doi.org/10.3390/drones9080539, URL https://www.mdpi.com/2504-446X/9/8/539.